

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCM-UTE

BÁO CÁO ĐỒ ÁN CUỐI KỲ
ĐỢT 2, HK II, NH 2026

ĐỀ TÀI: MÔ PHỎNG TÌM ĐƯỜNG ĐI TRONG KHÔNG GIAN 3D Ở
TRƯỜNG HỌC

GIÁO VIÊN HƯỚNG DẪN:

PGS.TS. Hoàng Văn Dũng

MÃ LỚP HỌC PHẦN:

ARIN330585_05

NHÓM THỰC HIỆN:

Nhóm 8

Sinh Viên Thực Hiện	MSSV
Cao Hoàng Phúc	24110303
Nguyễn Công Khoa	24110254
Phan Thị Thùy Linh	24110271

Thành phố Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

C

CHƯƠNG 1: MỞ ĐẦU.....	1
1.1. Lý do chọn đề tài.....	1
1.2. Mục tiêu đề tài.....	1
1.3. Đối tượng và phạm vi nghiên cứu.....	2
1.4. Phương pháp nghiên cứu.....	2
1.5. Ý nghĩa của đề tài.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	4
2.1. Ngôn ngữ lập trình.....	4
2.1.1. Giới thiệu về GDScript.....	4
2.1.2. Đặc điểm của ngôn ngữ.....	4
2.1.3. Giới thiệu về Python.....	5
2.2. Môi trường lập trình.....	5
2.3. Quy trình và phương pháp mô phỏng không gian 3D trường học.....	5
2.4. Cơ sở lý thuyết đồ thị trong không gian 3D.....	6
2.5. Hàm Heuristic trong không gian 3D.....	7
2.6. Nhóm thuật toán tìm kiếm mù (Uninformed Search).....	8
2.6.1. Thuật toán Breadth-First Search (BFS).....	8
2.6.2. Thuật toán Depth-First Search (DFS).....	9
2.6.3. Thuật toán Uniform Cost Search (UCS).....	9
2.7. Nhóm thuật toán tìm kiếm có định hướng (Informed Search).....	9
2.7.1. Greedy Best-First Search.....	10
2.7.2. Thuật toán A* (A-Star).....	10
2.8. Nhóm thuật toán tìm kiếm cục bộ và giới hạn bộ nhớ (Local Search).....	11
2.8.1. Thuật toán Hill Climbing (Leo đồi).....	11
CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG MÔ PHỎNG	12
3.1. Phân tích bài toán và Đề xuất giải pháp.....	12
3.1.1. Khó khăn của không gian 3D so với 2D.....	12
3.1.2. Giải pháp xử lý trọng số cầu thang.....	12

3.2. Thiết kế mô hình hệ thống (UML).....	13
3.2.1. Sơ đồ kiến trúc hệ thống (System Architecture Diagram).....	13
3.2.2. Sơ đồ Use Case (Use Case Diagram).....	15
3.2.3. Sơ đồ hoạt động (Activity Diagram).....	16
3.3. Tổ chức dữ liệu không gian trong GDScript.....	17
3.3.1. Cấu trúc lưu trữ Waypoint (Đỉnh).....	17
3.3.2. Biểu diễn đồ thị (Danh sách kề).....	20
3.4. Xây dựng và tinh chỉnh 6 thuật toán tìm đường.....	21
3.4.1. Triển khai nhóm tìm kiếm mù (BFS, DFS, UCS).....	21
3.4.2. Triển khai nhóm tìm kiếm có định hướng (Greedy, A*).....	24
3.4.3. Triển khai Random-Restart Hill Climbing.....	25
CHƯƠNG 4: THỰC NGHIỆM, ĐÁNH GIÁ, PHÂN TÍCH KẾT QUẢ.....	28
4.1. Xây dựng giao diện ứng dụng.....	28
4.2. Hướng dẫn thực thi phần mềm mô phỏng.....	31
4.2.1. Yêu cầu hệ thống.....	31
4.2.2. Các bước thực hiện.....	31
4.2.3. Bảng phím tắt.....	33
4.3. Kết quả sau khi chạy thuật toán.....	34
4.3.1. Kết quả thuật toán tìm kiếm mù (BFS, DFS, UCS).....	34
4.3.2. Kết quả thuật toán tìm kiếm có định hướng (Greedy, A*).....	37
4.3.3. Kết quả thuật toán tìm kiếm cục bộ (Hill Climbing).....	39
4.4. So sánh các thuật toán.....	40
4.4.1. Kịch bản 1 — Di chuyển cùng tầng.....	40
4.4.2. Kịch bản 2 — Di chuyển xuyên tầng.....	41
4.4.3. Nhận xét tổng hợp.....	43
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	45
5.1. Kết quả đạt được.....	45
5.2. Hạn chế của đề tài.....	46
5.3. Định hướng phát triển.....	47
TÀI LIỆU THAM KHẢO.....	48

DANH MỤC HÌNH ẢNH

Hình 3.1: Sơ đồ khối thể hiện luồng xử lý và tương tác giữa các bộ phận.....	13
Hình 3.2: Use Case Diagram: Hệ thống Mô phỏng Tìm Đường AI.....	15
Hình 3.3: Activity Diagram: Luồng Xử Lý Tìm Đường AI.....	16
Hình 4.1: Menu chọn thuật toán.....	28
Hình 4.2: Màn hình 3D sau khi ấn menu.....	29
Hình 4.3: Minh họa giao diện trực quan hóa khi thuật toán DFS đang được thực thi trong chế độ Running All.....	31
Hình 4.4: Chọn thuật toán đường đi.....	32
Hình 4.5: Giao diện hiển thị thông số Debug (Debug Overlay) khi thuật toán hoạt động.....	33
Hình 4.6: Kết quả thực thi thuật toán BFS.....	34
Hình 4.7: Kết quả thực thi thuật toán DFS.....	35
Hình 4.8: Kết quả thực thi thuật toán UCS.....	36
Hình 4.9: Kết quả thực thi thuật toán Greedy Best-First Search.....	37
Hình 4.10: Kết quả thực thi thuật toán A*.....	38
Hình 4.11: Hill Climbing không tìm được đường đi đến đích.....	39
Hình 4.12: Bảng so sánh hiệu năng 6 thuật toán: Kịch bản di chuyển cùng tầng.....	40
Hình 4.13: Bảng so sánh hiệu năng 6 thuật toán: Kịch bản di chuyển xuyên tầng.....	42

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án với đề tài “Mô phỏng tìm đường đi trong không gian 3D ở trường học”, nhóm chúng em đã nhận được sự quan tâm, hướng dẫn và hỗ trợ quý báu từ nhiều cá nhân và tập thể.

Trước hết, nhóm xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS. Hoàng Văn Dũng, giảng viên hướng dẫn, người đã tận tình chỉ bảo, định hướng và đóng góp nhiều ý kiến chuyên môn quý báu trong suốt quá trình thực hiện đề tài. Những kiến thức, kinh nghiệm và sự hỗ trợ của thầy đã giúp nhóm hoàn thành đồ án một cách tốt nhất.

Nhóm cũng xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh đã truyền đạt những kiến thức nền tảng và tạo điều kiện thuận lợi cho chúng em trong quá trình học tập và nghiên cứu.

Cuối cùng, nhóm xin cảm ơn các thành viên trong nhóm đã luôn đoàn kết, hỗ trợ lẫn nhau và cùng nỗ lực hoàn thành các nhiệm vụ được giao để hoàn thiện đồ án này.

Mặc dù đã cố gắng hết sức, nhưng do thời gian, kinh nghiệm và kiến thức còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn.

Xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 6 năm 2026

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong bối cảnh các cơ sở giáo dục ngày càng mở rộng về quy mô, hạ tầng trường học hiện đại thường bao gồm nhiều tòa nhà, nhiều tầng lầu với hệ thống giảng đường và phòng chức năng chồng chéo, phức tạp. Điều này đặt ra thách thức thực tế cho sinh viên, giảng viên và khách tham quan trong việc xác định và tìm kiếm lộ trình di chuyển hiệu quả giữa các không gian khác nhau trong khuôn viên trường.

Các giải pháp bản đồ 2D truyền thống ngày càng bộc lộ hạn chế khi không thể phản ánh chính xác mối quan hệ không gian theo chiều đứng (tầng lầu, cầu thang, thang máy), dẫn đến xu hướng chuyển dịch sang các giải pháp mô phỏng không gian trực quan bằng bản đồ 3D. Song song với đó, Trí tuệ nhân tạo (AI) cung cấp nhiều nhóm thuật toán tìm kiếm đường đi với đặc tính hiệu năng khác nhau, nhưng vẫn còn thiếu một môi trường thực nghiệm trực quan để kiểm chứng và đánh giá so sánh các thuật toán này trên cùng một bài toán thực tế.

Xuất phát từ những lý do trên, nhóm đề xuất thực hiện đề tài xây dựng hệ thống mô phỏng tìm đường trong không gian 3D tại môi trường trường học, kết hợp triển khai và đánh giá thực nghiệm các nhóm thuật toán tìm kiếm phổ biến trong AI.

1.2. Mục tiêu đề tài

- **Mục tiêu tổng quát:** Xây dựng ứng dụng mô phỏng không gian 3D trường học và tích hợp công cụ kiểm thử, so sánh trực quan hiệu năng của các thuật toán tìm đường AI.
- **Mục tiêu cụ thể:**
 - Xây dựng mô hình 3D trực quan của khuôn viên một trường học cụ thể, bao gồm các tòa nhà, tầng lầu, hành lang và cầu thang, phản ánh đúng cấu trúc không gian thực tế.

- Hiện thực hóa thành công 6 thuật toán tìm kiếm: BFS, DFS, UCS, Greedy, A*, Hill Climbing bằng ngôn ngữ GDScript trên nền tảng Godot Engine v4.5.2.
- Xây dựng hệ thống so sánh các thuật toán dựa trên các chỉ số thực nghiệm đo lường trực tiếp từ hệ thống mô phỏng, bao gồm: thời gian tính toán, số lượng node đã duyệt, độ dài đường đi và mức độ tối ưu của lộ trình.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài tập trung vào hai mảng chính:

- Lý thuyết đồ thị và các giải thuật tìm kiếm theo ba nhóm: Các thuật toán được chia thành ba nhóm: Uninformed Search (BFS, DFS, UCS), Informed Search (Greedy, A*), và Local Search (Hill Climbing).
- Mô hình mạng lưới giao thông dạng Waypoint/Node Network trong không gian 3D của trường học, làm cơ sở biểu diễn bài toán tìm đường dưới dạng đồ thị có trọng số.

Phạm vi nghiên cứu được giới hạn trong khuôn viên một số tòa nhà trọng điểm của trường học giả lập, bao gồm các phòng học, hành lang và cầu thang bộ, đủ để tạo ra các kịch bản di chuyển đa dạng phục vụ mục đích đánh giá thuật toán.

1.4. Phương pháp nghiên cứu

Đề tài sử dụng kết hợp hai phương pháp nghiên cứu bổ trợ cho nhau:

Phương pháp lý thuyết: Nghiên cứu tài liệu, phân tích cơ sở toán học, nguyên lý hoạt động, cũng như ưu điểm và nhược điểm của 6 giải thuật tìm kiếm được lựa chọn. Kết quả của giai đoạn này làm nền tảng lý luận cho toàn bộ hệ thống.

Phương pháp thực nghiệm: Triển khai lập trình thực tế hệ thống tìm đường trên môi trường mô phỏng 3D, tiến hành chạy thử nghiệm nhiều kịch bản di chuyển trong trường học giả lập (di chuyển cùng tầng, di chuyển xuyên tầng), từ đó thu thập số liệu định lượng và phân tích, đánh giá so sánh hiệu năng các thuật toán.

1.5. Ý nghĩa của đề tài

Ý nghĩa học thuật: Đề tài góp phần hệ thống hóa lý thuyết về các giải thuật tìm kiếm từ mức cơ bản (BFS, DFS) đến nâng cao (A*, Hill Climbing), đồng thời minh họa trực quan cách các thuật toán này vận hành trong không gian ba chiều — một bối cảnh thực tế hơn so với các bài toán đồ thị phẳng truyền thống.

Ý nghĩa thực tiễn: Hệ thống cung cấp giải pháp tiềm năng hỗ trợ tân sinh viên, giảng viên và khách tham quan dễ dàng định vị và tìm đường đi tối ưu giữa các phòng học đa tầng thông qua giao diện game tương tác trực quan.

Giá trị đánh giá thuật toán AI: Đề tài tạo ra một môi trường thử nghiệm chuẩn hóa, cho phép so sánh khách quan các yếu tố hiệu năng của nhiều thuật toán tìm kiếm trên cùng một cấu trúc dữ liệu đồ thị thực tế, qua đó cung cấp cơ sở thực nghiệm có giá trị tham khảo cho các nghiên cứu tìm đường trong không gian 3D.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Ngôn ngữ lập trình

2.1.1. Giới thiệu về GDScript

GDScript là ngôn ngữ lập trình kịch bản (scripting language) được Godot Engine phát triển và tích hợp riêng, đóng vai trò là ngôn ngữ lập trình chính thức của nền tảng này. Cú pháp của GDScript được thiết kế theo hướng tiếp cận gần gũi với Python — sử dụng thụt lề để phân cấp khối lệnh, kiểu gõ động (dynamic typing) và hỗ trợ kiểu gõ tĩnh tùy chọn (static typing) — giúp giảm đáng kể thời gian làm quen cho người học lập trình game lần đầu.

Trong dự án, GDScript được sử dụng để lập trình toàn bộ logic tìm đường của 6 giải thuật AI, quản lý hành vi di chuyển của nhân vật trong không gian 3D, và điều phối hệ thống hiển thị kết quả so sánh hiệu năng trực tiếp trên giao diện.

2.1.2. Đặc điểm của ngôn ngữ

GDScript sở hữu một số đặc điểm kỹ thuật nổi bật phù hợp với yêu cầu của dự án:

Tích hợp chặt chẽ với Godot Engine: Mỗi Script GDScript được gắn trực tiếp với một Node trong cây cảnh (Scene Tree), cho phép tổ chức mã nguồn của 6 giải thuật thành các module độc lập, dễ dàng tái sử dụng và mở rộng mà không phát sinh sự phụ thuộc chéo giữa các thuật toán.

Quản lý bộ nhớ tự động: GDScript áp dụng cơ chế thu gom rác (Garbage Collection) kết hợp với Reference Counting, cho phép tự động giải phóng bộ nhớ của các nút đồ thị tạm thời (frontier nodes, visited sets) được tạo ra trong quá trình mở rộng tìm kiếm — điều đặc biệt quan trọng khi hệ thống xử lý đồ thị có hàng trăm waypoint.

Hỗ trợ cấu trúc dữ liệu mạnh mẽ: GDScript cung cấp sẵn các kiểu cấu trúc dữ liệu cốt lõi như Array (mảng động), Dictionary (bảng băm) và PriorityQueue (thông qua thư viện tích hợp), đáp ứng trực tiếp yêu cầu cài đặt hàng đợi, ngăn xếp và hàng đợi ưu tiên của các giải thuật tìm kiếm trên đồ thị 3D.

2.1.3. Giới thiệu về Python

Bên cạnh GDScript, Python được sử dụng ở giai đoạn tiền xử lý đồ họa thông qua Blender Python API (bpy). Python đóng vai trò tự động hóa quá trình tạo và xuất mô hình 3D của tòa nhà trường học: thay vì dựng từng phòng học, hành lang và cầu thang theo cách thủ công, các script Python được viết để tạo lặp lại các khối hình học với thông số nhất quán (kích thước, vị trí, hướng xoay), từ đó xuất ra file .glb để import vào Godot Engine. Phương pháp này đảm bảo tính chính xác về tỷ lệ không gian và tiết kiệm đáng kể thời gian xây dựng mô hình.

2.2. Môi trường lập trình

Dự án sử dụng *Godot Engine phiên bản v4.5.2* làm nền tảng phát triển chính. *Godot v4.5.2* được lựa chọn vì cung cấp đầy đủ các công cụ cần thiết cho bài toán mô phỏng không gian 3D: hệ thống quản lý không gian ba chiều tích hợp, xử lý va chạm vật lý thông qua các thành phần *CollisionShape3D*, quản lý luồng vẽ đồ họa bằng *MeshInstance3D* và điều khiển nhân vật AI di chuyển có định hướng thông qua kiến trúc *Node-Script* linh hoạt.

Ngoài ra, *Blender* (kết hợp với Python API) được sử dụng như một công cụ hỗ trợ chuyên biệt cho giai đoạn tạo tài nguyên 3D, trước khi các tài nguyên này được chuyển sang Godot để tích hợp vào hệ thống mô phỏng.

2.3. Quy trình và phương pháp mô phỏng không gian 3D trường học

Quy trình xây dựng môi trường mô phỏng trường học trong dự án được thực hiện theo hai giai đoạn liên tiếp:

Giai đoạn tạo mô hình (Blender): Các tòa nhà, phòng học, hành lang và cầu thang được dựng hình trong Blender bằng cách kết hợp thao tác trực tiếp và *Python Script* để tự động hóa việc tạo các cấu trúc lặp lại. Sau khi hoàn thiện, mô hình được xuất sang định dạng .glb — một chuẩn file 3D mở, tương thích hoàn toàn với *Godot Engine* — rồi import vào dự án. Hệ thống cầu thang bộ và thang máy được thiết lập với các vùng va chạm *CollisionShape3D* chính xác, cho phép nhân vật di chuyển qua lại giữa các tầng lầu theo trục Y một cách tự nhiên.

Hệ thống Camera đa góc nhìn: Để phục vụ cả trải nghiệm người dùng lẫn nhu cầu quan sát thuật toán, hệ thống trang bị hai chế độ camera:

- Agent-view Camera: Bám theo nhân vật AI dưới góc nhìn người thứ ba, cho phép người dùng quan sát trực tiếp lộ trình di chuyển trong không gian 3D như thể đang trải nghiệm thực tế.
- Top-down Camera: Góc nhìn từ trên cao bao quát toàn bộ mặt bằng từng tầng lầu, giúp quan sát rõ ràng toàn bộ đường đi mà thuật toán vẽ ra — đặc biệt hữu ích khi so sánh lộ trình giữa các giải thuật khác nhau.

2.4. Cơ sở lý thuyết đồ thị trong không gian 3D

Để các giải thuật tìm kiếm AI có thể hoạt động trên môi trường trường học, toàn bộ mạng lưới giao thông trong khuôn viên được số hóa và mô hình hóa thành đồ thị có trọng số $G = (V, E)$, trong đó:

Đỉnh (Node/Waypoint) V là các đối tượng Node3D trong không gian Godot, mỗi đỉnh lưu trữ tọa độ ba chiều thực tế dưới dạng Vector3(x, y, z). Các waypoint được đặt tại những vị trí có ý nghĩa giao thông: cửa ra vào phòng học, hành lang và cầu thang bộ — đảm bảo mạng lưới phản ánh đúng các lựa chọn di chuyển thực tế.

Cạnh (Edge) E là đường nối trực tiếp giữa hai waypoint kề nhau có thể đi lại được, biểu diễn một đoạn đường đi thực tế trong khuôn viên (đoạn hành lang, một bậc thang, v.v.).

Trọng số W của mỗi cạnh được tính theo khoảng cách Euclid ba chiều giữa hai nút:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Để phản ánh chi phí leo tầng trong môi trường 3D, hệ thống áp dụng cơ chế phạt độ cao (ascent penalty). Trọng số cạnh được xác định bằng tổng khoảng cách Euclid và phần chi phí bổ sung tỷ lệ với độ chênh lệch cao độ giữa hai waypoint.

$$w(u, v) = 1.0 * d(u, v) + \lambda * ascent(u, v)$$

Trong đó:

- $d(u, v)$: khoảng cách Euclid
- $ascent(u, v)$: phần tăng độ cao
- $\lambda = 4.0$

2.5. Hàm Heuristic trong không gian 3D

Trước khi đi vào nhóm thuật toán tìm kiếm có định hướng, cần làm rõ khái niệm nền tảng mà các thuật toán này sử dụng: hàm heuristic.

Định nghĩa: Hàm heuristic $h(n)$ là hàm ước lượng chi phí còn lại từ đỉnh hiện tại n đến đích mà không cần thực hiện quá trình tìm kiếm đầy đủ trên đồ thị. Một hàm heuristic được gọi là chấp nhận được (admissible) nếu giá trị ước lượng của nó không bao giờ vượt quá chi phí thực tế tối ưu từ n đến đích, tức là:

$$h(n) \leq h^*(n)$$

, trong đó $h^*(n)$ là chi phí thực tế nhỏ nhất từ đỉnh n đến đích. Khi sử dụng một heuristic admissible, thuật toán A* được đảm bảo tìm ra đường đi tối ưu.

Lựa chọn heuristic cho dự án: Trong đồ thị $G = (V, E)$ được xây dựng từ mạng lưới waypoint của trường học, mỗi đỉnh lưu tọa độ thực tế dưới dạng Vector3(x, y, z). Vì vậy, hàm heuristic được lựa chọn là khoảng cách Euclid ba chiều giữa đỉnh hiện tại n và đích g :

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2 + (z_g - z_n)^2}$$

Khoảng cách Euclid 3D phản ánh khoảng cách đường thẳng ngắn nhất giữa hai vị trí trong không gian. Hàm heuristic này phù hợp với môi trường mô phỏng vì có chi phí tính toán thấp, đồng thời cung cấp thông tin định hướng hiệu quả cho các thuật toán tìm kiếm.

Tính admissible của heuristic: Trong hệ thống, chi phí thực tế của mỗi cạnh được xác định bằng khoảng cách Euclid kết hợp với phần chi phí bổ sung cho sự thay đổi độ cao giữa hai waypoint. Do đó, chi phí thực tế của mọi lộ trình luôn lớn hơn hoặc bằng khoảng cách đường thẳng nối trực tiếp từ vị trí hiện tại đến đích. Nói cách khác, khoảng cách Euclid 3D không bao giờ đánh giá cao hơn chi phí thực tế tối ưu:

$$h(n) \leq h^*(n)$$

Vì vậy, heuristic được sử dụng trong đề tài là admissible và phù hợp cho thuật toán A*.

Đối với thuật toán A*, hàm đánh giá tổng hợp được xác định như sau:

$$f(n) = g(n) + h(n)$$

trong đó:

- $g(n)$: chi phí thực tế đã tích lũy từ đỉnh xuất phát đến đỉnh n .
- $h(n)$: chi phí ước lượng từ đỉnh n đến đỉnh đích dựa trên khoảng cách Euclid 3D.

Sự kết hợp giữa $g(n)$ và $h(n)$ cho phép A^* vừa xem xét chi phí đã đi, vừa định hướng quá trình tìm kiếm về phía đích, từ đó đạt được sự cân bằng giữa tốc độ tìm kiếm và tính tối ưu của lời giải.

2.6. Nhóm thuật toán tìm kiếm mù (Uninformed Search)

Nhóm thuật toán tìm kiếm mù không sử dụng thông tin về vị trí đích mà chỉ dựa trên cấu trúc đồ thị để mở rộng các nút. Trong đề tài, nhóm thuật toán này được sử dụng để đánh giá sự khác biệt giữa các chiến lược tìm kiếm theo bề rộng, chiều sâu và chi phí tích lũy.

2.6.1. Thuật toán Breadth-First Search (BFS)

BFS mở rộng các đỉnh theo từng lớp (level), sử dụng hàng đợi FIFO (First-In First-Out): mỗi đỉnh được thêm vào cuối hàng đợi và lấy ra từ đầu, đảm bảo các đỉnh gần nguồn luôn được xét trước. Trong đồ thị waypoint của dự án, BFS sẽ lần lượt duyệt tất cả các nút cùng số bước từ điểm xuất phát trước khi tiến sâu hơn.

- **Ưu điểm:** Đảm bảo tìm được đường đi ngắn nhất tính theo số cạnh, phù hợp với các kịch bản di chuyển trong cùng một tầng lầu nơi các cạnh có độ dài tương đương nhau.
- **Nhược điểm:** Trong môi trường trường học nhiều tầng với hàng trăm waypoint, BFS có thể phải lưu trữ đồng thời toàn bộ biên mở rộng (frontier) của một lớp — gây áp lực bộ nhớ đáng kể, đặc biệt ở các tầng có mật độ waypoint cao.

2.6.2. Thuật toán Depth-First Search (DFS)

DFS mở rộng đỉnh theo chiều sâu bằng cách sử dụng ngăn xếp LIFO (Last-In First-Out): đỉnh được thêm vào đỉnh ngăn xếp và lấy ra từ đỉnh, khiến thuật toán luôn ưu tiên đi sâu theo một nhánh trước khi quay lui. Trong đồ thị trường học, DFS có xu hướng đi xuyên qua nhiều tầng lầu liên tiếp hoặc dọc theo một hành lang đến cùng trước khi thử hướng khác.

- **Ưu điểm:** Yêu cầu bộ nhớ thấp — tại mỗi thời điểm chỉ cần lưu trữ một nhánh duyệt duy nhất từ gốc đến đỉnh hiện tại, thay vì toàn bộ biên mở rộng như BFS.
- **Nhược điểm:** Không đảm bảo đường đi ngắn nhất — trong môi trường 3D với nhiều nhánh đồ thị phức tạp, đường đi DFS tìm được thường mang tính "zigzag", vòng vèo qua nhiều phòng học và hành lang không liên quan trước khi đến đích. Đặc biệt, DFS không phù hợp cho bài toán định tuyến thực tế vì không có cơ chế phân biệt chi phí giữa các loại di chuyển.

2.6.3. Thuật toán Uniform Cost Search (UCS)

UCS mở rộng đỉnh theo chi phí tích lũy thực tế $g(n)$ bằng cách sử dụng hàng đợi ưu tiên (Priority Queue): tại mỗi bước, đỉnh có tổng chi phí đường đi nhỏ nhất từ nguồn được ưu tiên mở rộng trước. UCS về bản chất là phiên bản tổng quát hóa của BFS, hoạt động đúng đắn trên đồ thị có trọng số không âm.

- **Ưu điểm:** Đảm bảo tìm được đường đi có tổng chi phí thực tế nhỏ nhất đã thiết kế ở mục 2.4. Đây là thuật toán mù duy nhất trong nhóm có khả năng tối ưu chi phí trên đồ thị trọng số của dự án.
- **Nhược điểm:** Không có thông tin định hướng về vị trí đích, nên UCS mở rộng nút theo mọi hướng đồng đều — dẫn đến số lượng nút duyệt qua có thể rất lớn trong môi trường trường học nhiều tầng rộng lớn trước khi tìm đến đích.

2.7. Nhóm thuật toán tìm kiếm có định hướng (Informed Search)

Khác với nhóm tìm kiếm mù, nhóm thuật toán có định hướng khai thác hàm heuristic $h(n)$ đã trình bày ở mục 2.5 để dẫn hướng quá trình tìm kiếm về phía đích. Điều này cho phép giảm đáng kể số lượng nút cần duyệt trong không gian đồ thị lớn như môi trường trường học nhiều tầng của dự án.

2.7.1. Greedy Best-First Search

Greedy Best-First Search ưu tiên mở rộng đỉnh dựa hoàn toàn vào $h(n)$ — tức là khoảng cách Euclid 3D từ đỉnh hiện tại đến đích — mà không tính đến chi phí đường đi đã tích lũy

$g(n)$. Tại mỗi bước, thuật toán luôn chọn đỉnh có vẻ "gần đích nhất" theo ước lượng heuristic.

- **Ưu điểm:** Tốc độ tìm kiếm nhanh, số nút mở rộng ít hơn so với các thuật toán mù, đặc biệt hiệu quả trong những kịch bản di chuyển không có vật cản phức tạp giữa điểm xuất phát và đích.
- **Nhược điểm:** Trong môi trường trường học, khi điểm xuất phát và điểm đích nằm ở hai phòng học liền kề nhau nhưng ngăn cách bởi tường, Greedy có thể bị kẹt khi liên tục chọn các waypoint "gần nhất về đường chim bay" mà thực tế không thể đi qua. Ngoài ra, thuật toán không đảm bảo đường đi tối ưu vì bỏ qua hoàn toàn chi phí đã tích lũy.

2.7.2. Thuật toán A* (A-Star)

A* là sự kết hợp có hệ thống giữa UCS và Greedy Best-First Search. Thay vì chỉ xét $g(n)$ (như UCS) hoặc chỉ xét $h(n)$ (như Greedy), A* đánh giá mỗi đỉnh thông qua hàm tổng hợp:

$$f(n) = g(n) + h(n)$$

, trong đó $g(n)$ là chi phí thực tế tích lũy (tính theo trọng số cạnh đã nhân hệ số cản k và $h(n)$ là khoảng cách Euclid 3D đến đích. Hàng đợi ưu tiên sắp xếp các đỉnh theo $f(n)$ tăng dần, đảm bảo đỉnh được mở rộng tiếp theo là đỉnh hứa hẹn nhất xét cả về chi phí đã đi lẫn ước lượng còn lại.

- **Ưu điểm:** Đảm bảo tìm được đường đi tối ưu về chi phí (tổng trọng số nhỏ nhất) khi hàm heuristic chấp nhận được — điều đã được chứng minh với hàm Euclid 3D sử dụng trong dự án. A* vừa tránh được sự lãng phí của UCS (duyệt theo mọi hướng) vừa tránh được sự thiếu chính xác của Greedy (bỏ qua chi phí đã đi).
- **Nhược điểm:** Hiệu năng phụ thuộc vào chất lượng của hàm heuristic; trong một số kịch bản phức tạp, A* vẫn có thể phải mở rộng một lượng lớn nút trước khi xác định được đường tối ưu.

2.8. Nhóm thuật toán tìm kiếm cục bộ và giới hạn bộ nhớ (Local Search)

Khác với hai nhóm trên vốn duy trì danh sách các đỉnh chờ mở rộng, nhóm thuật toán này hoạt động với lượng thông tin được lưu giữ có chủ đích giới hạn — hoặc chỉ xét các đỉnh lân cận tức thời (Hill Climbing). Điều này mang lại lợi thế về bộ nhớ nhưng đồng thời làm mất đi sự đảm bảo về tính tối ưu.

2.8.1. Thuật toán Hill Climbing (Leo đồi)

Trong triển khai thực tế, nhóm sử dụng biến thể Random-Restart Hill Climbing: nếu thuật toán rơi vào cực tiểu cục bộ, hệ thống tự động khởi động lại từ một đỉnh lân cận được chọn ngẫu nhiên trong nhóm ứng viên tốt nhất, với tối đa 6 lần khởi động lại. Cơ chế này giúp tăng khả năng thoát khỏi cực tiểu cục bộ so với Hill Climbing đơn thuần, tuy nhiên vẫn không đảm bảo tìm được lời giải tối ưu toàn cục.

- **Ưu điểm:** Tiêu thụ bộ nhớ cực kỳ thấp — tại mọi thời điểm chỉ cần lưu trạng thái của đỉnh hiện tại.
- **Nhược điểm:** Dễ rơi vào cực tiểu cục bộ (local minimum) — tình huống mà tất cả các đỉnh kề đều có $h(n)$ lớn hơn đỉnh hiện tại, khiến thuật toán dừng lại mà chưa đến đích. Trong môi trường trường học, kịch bản này xảy ra điển hình khi nhân vật đứng ở cuối hành lang tầng 1 ngay bên dưới phòng đích ở tầng 2 — hàm Euclid 3D cho thấy vị trí hiện tại đã rất gần đích, nhưng không có đỉnh kề nào gần hơn vì con đường thực tế phải vòng ra chiều nghiêng cầu thang. Thuật toán không có cơ chế quay lui để thoát khỏi tình huống này.

CHƯƠNG 3: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG MÔ PHỎNG

3.1. Phân tích bài toán và Đề xuất giải pháp

3.1.1. Khó khăn của không gian 3D so với 2D

Trong bài toán tìm đường trên mặt phẳng 2D, mọi vị trí được xác định bởi hai tọa độ và việc di chuyển giữa các điểm diễn ra trên cùng một mặt phẳng liên tục. Môi trường trường học 3D phá vỡ giả định này theo ba hướng:

Thứ nhất, sự xuất hiện của trục Y (chiều cao) tạo ra tình huống mà hai waypoint có thể rất gần nhau theo khoảng cách Euclid nhưng lại không có cạnh kết nối trực tiếp — nhân vật bắt buộc phải di chuyển vòng ra cầu thang mới có thể đổi tầng. Điều này khiến khoảng cách hình học và chi phí di chuyển thực tế không còn tương đồng, ảnh hưởng trực tiếp đến độ chính xác của các thuật toán dựa trên heuristic.

Thứ hai, môi trường đa tầng làm tăng đáng kể kích thước đồ thị do phải bổ sung các waypoint trung gian tại chiều ngẩng cầu thang và điểm giao tầng, kéo theo áp lực bộ nhớ và thời gian duyệt lớn hơn so với bài toán 2D tương đương.

Thứ ba, các thuật toán cục bộ như Hill Climbing đặc biệt dễ rơi vào cực tiểu cục bộ trong không gian 3D: khi nhân vật đứng ngay dưới phòng đích ở tầng trên, mọi bước di chuyển ngang ra cầu thang đều làm tăng $h(n)$, khiến thuật toán từ chối thực hiện và bị kẹt tại chỗ.

3.1.2. Giải pháp xử lý trọng số cầu thang

Để phản ánh sự khác biệt về nỗ lực di chuyển giữa các loại đường đi, trọng số w của mỗi cạnh trong đồ thị được tính theo công thức:

$$w(u, v) = 1.0 * d(u, v) + \lambda * ascent(u, v)$$

với $\lambda = 4.0$

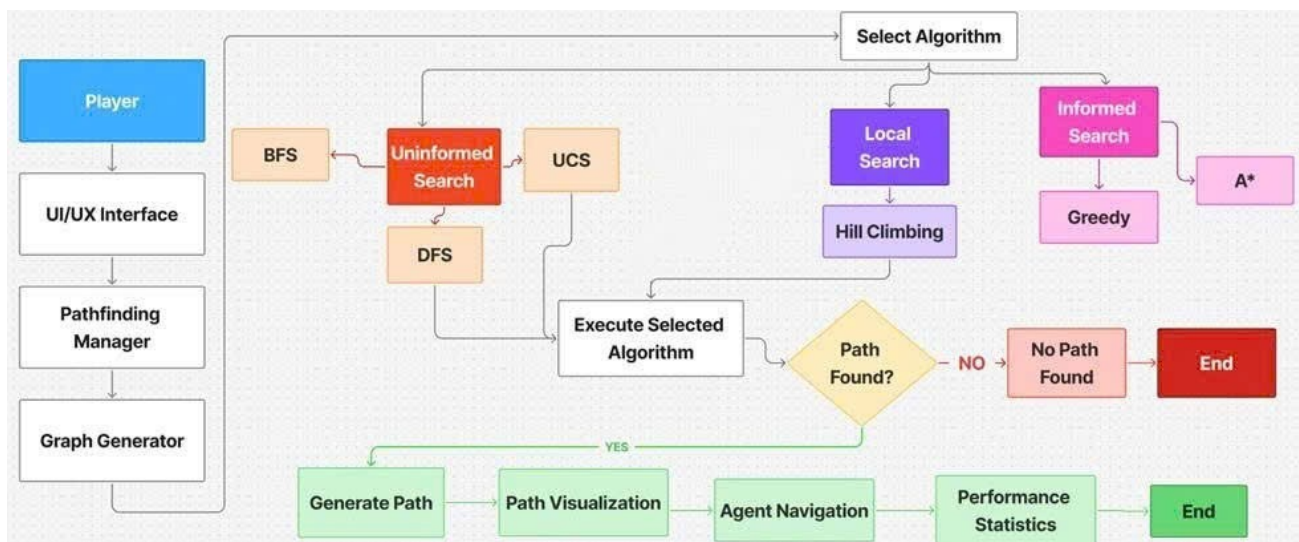
- đi ngang $\rightarrow$ ascent = 0
- đi lên $\rightarrow$ bị phạt thêm
- đi xuống $\rightarrow$ không bị phạt

Cơ chế này có ý nghĩa trực tiếp đối với các thuật toán tối ưu chi phí: UCS và A* sẽ tự nhiên ưu tiên lộ trình đi qua hành lang và chỉ sử dụng cầu thang khi không còn phương án nào có tổng w nhỏ hơn. Đồng thời, sự phân hóa trọng số tạo ra các kịch bản thực nghiệm có ý nghĩa để so sánh hiệu năng giữa sáu thuật toán, đặc biệt trong các tình huống di chuyển xuyên tầng.

3.2. Thiết kế mô hình hệ thống (UML)

3.2.1. Sơ đồ kiến trúc hệ thống (System Architecture Diagram)

Sơ đồ kiến trúc hệ thống mô tả luồng xử lý tổng thể từ thao tác của người dùng trên giao diện cho đến khi kết quả được hiển thị lên màn hình 3D (xem Hình 3.1).



Hình 3.1: Sơ đồ khối thể hiện luồng xử lý và tương tác giữa các bộ phận

Như thể hiện trong Hình 3.1, hệ thống được tổ chức theo mô hình pipeline tuyến tính kết hợp phân nhánh, trong đó mỗi thành phần đảm nhận một vai trò chuyên biệt và chuyển giao dữ liệu cho thành phần tiếp theo. Cụ thể, luồng xử lý diễn ra qua sáu giai đoạn chính:

Giai đoạn 1 — Tiếp nhận yêu cầu (UI/UX Interface):

Người dùng (Player) tương tác với hệ thống thông qua giao diện đồ họa, thực hiện các thao tác chọn điểm xuất phát, điểm đích và thuật toán mong muốn.

Giai đoạn 2 — Điều phối (Pathfinding Manager):

Thành phần trung tâm tiếp nhận yêu cầu từ giao diện, kiểm tra tính hợp lệ của tham số đầu vào và điều phối toàn bộ quy trình xử lý phía sau.

Giai đoạn 3 — Xây dựng đồ thị (Graph Generator):

Dựa trên dữ liệu không gian 3D của khuôn viên trường, Graph Generator khởi tạo mạng lưới Waypoint Graph với đầy đủ thông tin đỉnh, cạnh và trọng số trước khi chuyển sang bước thực thi thuật toán.

Giai đoạn 4 — Lựa chọn và thực thi thuật toán (Select Algorithm):

Pathfinding Manager phân nhánh yêu cầu sang đúng thuật toán được chọn thuộc một trong ba nhóm: nhóm tìm kiếm mù (Uninformed Search: BFS, DFS, UCS), nhóm tìm kiếm có định hướng (Informed Search: Greedy, A*) và nhóm tìm kiếm cục bộ (Local Search: Hill Climbing).

Giai đoạn 5 — Tổng hợp kết quả (Path Result):

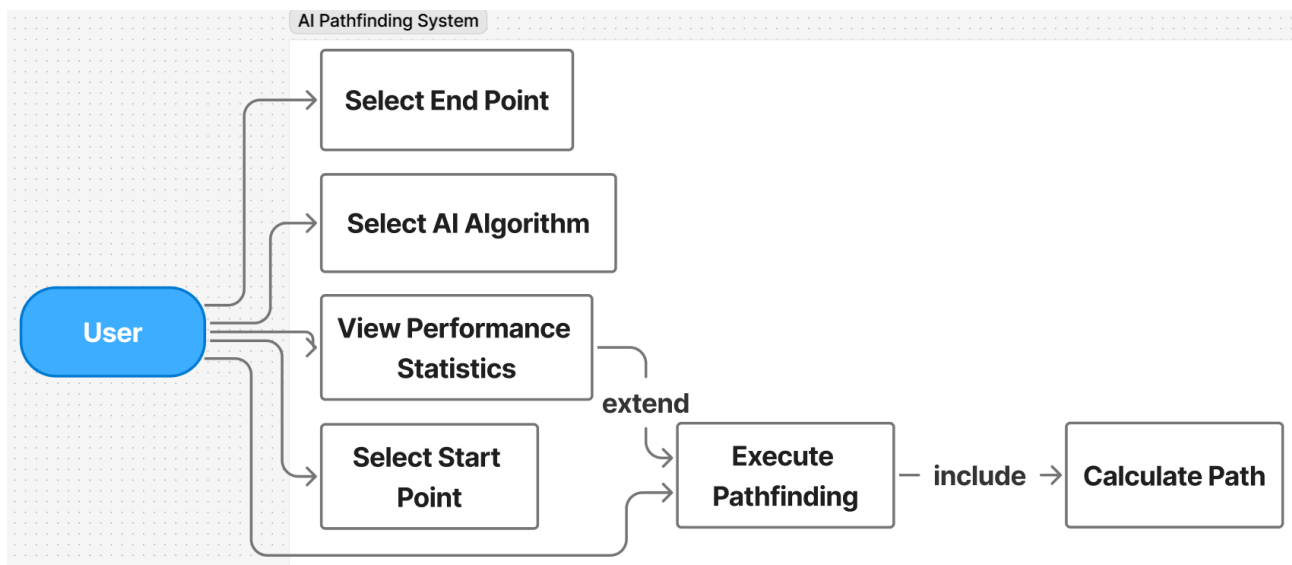
Sau khi thuật toán hoàn thành, kết quả tìm kiếm — bao gồm mảng nút tạo thành lộ trình và các chỉ số hiệu năng — được tổng hợp và chuẩn hóa thành định dạng thống nhất để chuyển sang giai đoạn hiển thị.

Giai đoạn 6 — Hiển thị lộ trình (3D Renderer):

Thành phần cuối cùng trong pipeline nhận mảng nút từ Path Result, vẽ lộ trình trực quan lên không gian 3D thông qua MeshInstance3D, đồng thời kích hoạt chuyển động của nhân vật Agent và hiển thị bảng thống kê hiệu năng trên giao diện.

3.2.2. Sơ đồ Use Case (Use Case Diagram)

Sơ đồ Use Case mô tả toàn bộ các chức năng mà người dùng có thể thực hiện trong hệ thống mô phỏng tìm đường AI (xem Hình 3.2).



Hình 3.2: Use Case Diagram: Hệ thống Mô phỏng Tìm Đường AI

Hệ thống có duy nhất một tác nhân là User (người dùng), tương tác trực tiếp với bốn chức năng chính trong phạm vi hệ thống AI Pathfinding System.

Select Start Point / Select End Point: Người dùng lựa chọn phòng xuất phát và phòng đích từ danh sách các waypoint có sẵn trong khuôn viên trường. Đây là bước khởi đầu bắt buộc, xác định bài toán đầu vào cho toàn bộ quá trình tìm đường. Hai chức năng này hoạt động độc lập nhau nhưng cùng cung cấp tham số không gian đầu vào cho quá trình thực thi thuật toán.

Select AI Algorithm: Người dùng chọn một trong sáu thuật toán tìm đường được tích hợp trong hệ thống — BFS, DFS, UCS (nhóm Uninformed Search), Greedy, A* (nhóm Informed Search), Hill Climbing (nhóm Local Search) hoặc chọn cả 6 thuật toán để so sánh hiệu suất. Lựa chọn này quyết định chiến lược tìm kiếm sẽ được áp dụng cho lần chạy mô phỏng tiếp theo.

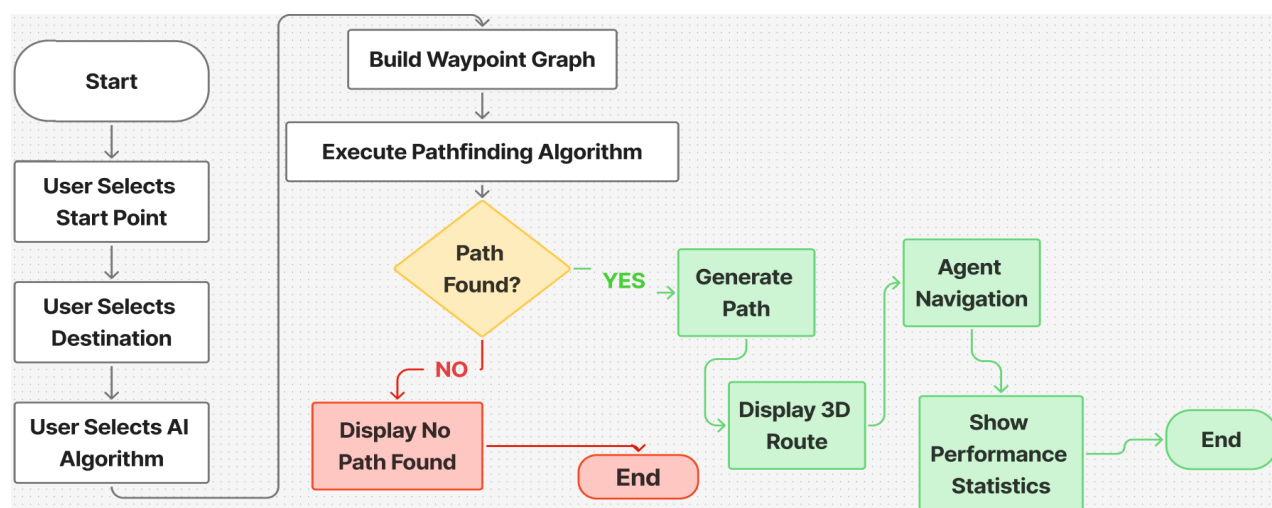
Execute Pathfinding: Sau khi người dùng đã thiết lập đủ điểm xuất phát, điểm đích và thuật toán, chức năng này kích hoạt toàn bộ quy trình tính toán và hiển thị lộ trình trực quan trong không gian 3D. Theo quan hệ «include» trong sơ đồ, Execute Pathfinding bắt

buộc gọi đến use case Calculate Path — thành phần chịu trách nhiệm thực thi thuật toán AI trên Waypoint Graph và trả về mảng nút tạo thành lộ trình.

View Performance Statistics: Chức năng này có quan hệ ‘extend’ với Execute Pathfinding, nghĩa là sau mỗi lần thực thi thành công, người dùng có thể tùy chọn xem bảng thống kê hiệu năng chi tiết của thuật toán vừa chạy, bao gồm thời gian tính toán, số node đã duyệt, độ dài đường đi tìm được và mức độ tối ưu so với lộ trình lý thuyết. Chức năng này là cơ sở để người dùng so sánh và đánh giá hiệu quả giữa các thuật toán trên cùng một kịch bản di chuyển.

3.2.3. Sơ đồ hoạt động (Activity Diagram)

Sơ đồ hoạt động mô tả luồng xử lý chi tiết từ thời điểm người dùng khởi động mô phỏng cho đến khi hệ thống hiển thị kết quả hoặc thông báo không tìm thấy đường đi (xem Hình 3.3).



Hình 3.3: Activity Diagram: Luồng Xử Lý Tìm Đường AI

Luồng hoạt động diễn ra tuần tự qua các bước sau:

Bước 1 — Thiết lập đầu vào: Từ trạng thái Start, người dùng lần lượt thực hiện ba thao tác khởi tạo: chọn điểm xuất phát (User Selects Start Point), chọn phòng đích (User Selects Destination) và chọn thuật toán AI mong muốn (User Selects AI Algorithm). Ba thao tác này diễn ra tuần tự và là điều kiện tiên quyết để hệ thống bắt đầu xử lý.

Bước 2 — Xây dựng đồ thị và thực thi thuật toán: Sau khi nhận đủ tham số đầu vào, hệ thống tự động xây dựng mạng lưới waypoint (Build Waypoint Graph) từ dữ liệu không gian 3D của khuôn viên trường, sau đó kích hoạt thuật toán AI đã được chọn (Execute Pathfinding Algorithm) trên đồ thị vừa dựng.

Bước 3 — Kiểm tra điều kiện "Path Found?": Đây là bước xác định hệ thống có tìm được đường đi hay không để tiếp tục xử lý theo nhánh tương ứng.

- **Nhánh YES — Tìm thấy đường đi:** Hệ thống tổng hợp mảng nút tạo thành lộ trình (Generate Path) → hiển thị lộ trình trực quan trong không gian 3D (Display 3D Route) → kích hoạt chuyển động của nhân vật Agent di chuyển theo lộ trình (Agent Navigation) → hiển thị bảng thống kê hiệu năng bao gồm thời gian tính toán, số node đã duyệt và độ dài đường đi (Show Performance Statistics) → End.
- **Nhánh NO — Không tìm thấy đường đi:** Hệ thống hiển thị thông báo lỗi (Display No Path Found) → End.

3.3. Tổ chức dữ liệu không gian trong GDScript

3.3.1. Cấu trúc lưu trữ Waypoint (Đỉnh)

Mỗi waypoint được lưu dưới dạng một dictionary trong file navigation_graph.json với các trường chính:

- **id** (int): định danh nguyên duy nhất của đỉnh.
- **position** (array): tọa độ 3D dạng [x, y, z] trong hệ tọa độ pipeline sinh dữ liệu.
- **kinds** (array): kiểu node, ví dụ "corridor", "stair_access", "stair_step".
- **labels** (array): nhãn mô tả vị trí cụ thể, ví dụ "Khoi_A.2_Phong_Hoc_Corridor_F0:2".
- **neighbors** (array[int]): danh sách id các đỉnh kề trực tiếp.

Ví dụ một node trích từ navigation_graph.json:

```
{  
  "id": 2,
```

```

"position": [-15.08, -87.8, 0.0],
"kinds": ["corridor", "stair_access"],
"labels": [
    "Khoi_A.2_Phong_Hoc_Corridor_F0:2",
    "Khoi_A.2_Phong_Hoc_StairCorridor_F0"
],
"neighbors": [1, 3, 5]
}

```

Dữ liệu này được sinh bởi generate_navigation_graph.py, trong đó mỗi node được khởi tạo qua hàm add_node():

```

def add_node(self, pos, kind, label):
    node_id = len(self.nodes)
    self.nodes.append({
        "id": node_id,
        "x": round(pos[0], 4),
        "y": round(pos[1], 4),
        "z": round(pos[2], 4),
        "kinds": [kind],
        "labels": [label],
    })
    self.adjacency[node_id] = set()
    return node_id

```

Quá trình khởi tạo bao gồm: sinh node mới hoặc gộp node nếu trùng vị trí; gán id, tọa độ, kinds, labels; duy trì adjacency[node_id] để theo dõi các đỉnh kề. Khi xuất file JSON, generator ghi node theo cấu trúc:

```

data["nodes"].append({

```

```

    "id": node["id"],
    "position": [node["x"], node["y"], node["z"]],
    "kinds": node["kinds"],
    "labels": node["labels"],
    "neighbors": neigh,
  })

```

Ánh xạ sang Vector3 trong GDScript: do Godot sử dụng trục Y hướng lên trong khi pipeline sinh dữ liệu dùng trục Z hướng lên, khi nạp JSON vào path_bot.gd, tọa độ được chuyển đổi như sau: `var pos_arr: Array = node_data["position"]`

```

var pos := Vector3(float(pos_arr[0]), float(pos_arr[2]), -float(pos_arr[1]))
_graph_positions[node_id] = pos
_graph_neighbors[node_id] = node_data.get("neighbors", [])

```

Trục Y của Godot lấy từ thành phần z của JSON, trục Z của Godot lấy từ -y của JSON — đảm bảo mô hình hiển thị đúng trong không gian 3D.

Suy ra tầng lầu (floor level): bộ dữ liệu không lưu trường floor_level tách biệt. Tuy nhiên, qua quan sát dữ liệu thực tế, thành phần cao độ pos.y sau ánh xạ nhận các giá trị rời rạc tương ứng với từng tầng:

- ☐ Tầng trệt: $y = 0.0$
- ☐ Tầng 1: $y = 4.0$
- ☐ Tầng 2: $y = 8.0$

Do đó, tầng lầu của một waypoint có thể suy ra bằng:

```

var floor_level := int( _graph_positions[node_id].y / 4.0)

```


3.3.2. Biểu diễn đồ thị (Danh sách kề)

Sau khi nạp JSON, `path_bot.gd` lưu đồ thị bằng hai dictionary:

- `_graph_positions`: ánh xạ `node_id` $\rightarrow$ `Vector3` — tọa độ không gian của từng đỉnh.
- `_graph_neighbors`: ánh xạ `node_id` $\rightarrow$ `Array[int]` — danh sách id các đỉnh kề.

Ví dụ với node 2 trong dữ liệu thực tế:

```
_graph_neighbors[2] = [1, 3, 5]
```

Khi thuật toán cần duyệt các đỉnh kề của node 2, chỉ cần lặp qua mảng này thay vì quét toàn bộ tập node.

Tính trọng số cạnh: chi phí di chuyển giữa hai đỉnh được tính bởi hàm `_distance_between()`:

```
const EDGE_LENGTH_WEIGHT := 1.0
const ASCENT_COST_WEIGHT := 4.0

func _distance_between(a: int, b: int) -> float:
    var from_pos: Vector3 = _graph_positions[a]
    var to_pos: Vector3 = _graph_positions[b]
    var edge_length: float = from_pos.distance_to(to_pos)
    var ascent_delta: float = to_pos.y - from_pos.y
    var ascent: float = maxf(0.0, ascent_delta)
    return edge_length * EDGE_LENGTH_WEIGHT + ascent *
ASCENT_COST_WEIGHT
```

Công thức gồm hai thành phần: khoảng cách Euclid nhân `EDGE_LENGTH_WEIGHT = 1.0`, cộng thêm phần phạt leo lên `ascent` ×

`ASCENT_COST_WEIGHT = 4.0`. Phần phạt chỉ áp dụng khi đỉnh đích cao hơn đỉnh nguồn ($\text{ascent} > 0$), tức chỉ tính chi phí leo lên, không phạt khi đi xuống. Cơ chế này khiến UCS và A* tự nhiên ưu tiên lộ trình đi ngang trước khi sử dụng cầu thang — cơ chế phạt độ cao (ascent penalty) được hiện thực trực tiếp trong hàm `_distance_between()`.

3.3.3. Lý do chọn danh sách kề thay vì ma trận kề

Bộ dữ liệu thực tế của dự án có `node_count = 6298` và `edge_count = 6746` — đồ thị rất thưa với số cạnh chỉ xấp xỉ số đỉnh. Nếu dùng ma trận kề kích thước 6298×6298 , phần lớn ô sẽ có giá trị 0, gây lãng phí bộ nhớ nghiêm trọng. Danh sách kề chỉ lưu các cạnh thực sự tồn tại, cho phép duyệt hàng xóm của một đỉnh trong $O(\deg(v))$ thay vì $O(N)$ — phù hợp với yêu cầu hiệu năng khi chạy cả 6 thuật toán tìm kiếm trên đồ thị lớn trong Godot Engine.

3.4. Xây dựng và tinh chỉnh 6 thuật toán tìm đường

3.4.1. Triển khai nhóm tìm kiếm mù (BFS, DFS, UCS)

Cấu trúc dữ liệu:

Ba thuật toán BFS, DFS và UCS dùng chung các biến frontier sau:

```
var _search_frontier: Array[int] = []
var _priority_frontier_heap: Array = []
var _frontier_membership: Dictionary = {}
var _frontier_queue_head := 0
```

- **_search_frontier:** mảng lưu id các node, dùng cho BFS và DFS.
- **_priority_frontier_heap:** binary min-heap lưu cặp `[priority, node_id]`, dùng riêng cho UCS.
- **_frontier_membership:** dictionary đánh dấu node còn trong frontier, tránh xử lý trùng lặp.

- `_frontier_queue_head`: con trỏ FIFO cho BFS — thay vì xóa phần tử đầu mảng tồn kém, hệ thống dịch chuyển con trỏ và compact mảng định kỳ.

Vòng lặp mở rộng node (trích từ `_step_search()`):

```
for neighbor_id_variant in _graph_neighbors.get(current_id, []):
    var neighbor_id := int(neighbor_id_variant)
    if _closed_membership.has(neighbor_id):
        continue
    match search_algorithm:
        SearchAlgorithm.BFS, SearchAlgorithm.DFS:
            if _frontier_membership.has(neighbor_id) or neighbor_id == _search_start_id or
            _came_from.has(neighbor_id):
                continue
            _came_from[neighbor_id] = current_id
            _g_score[neighbor_id] = float(_g_score.get(current_id, 0.0)) +
            _distance_between(current_id, neighbor_id)
            _push_frontier(neighbor_id)
        SearchAlgorithm.UCS:
            var tentative_g: float = float(_g_score.get(current_id, INF)) +
            _distance_between(current_id, neighbor_id)
            if tentative_g < float(_g_score.get(neighbor_id, INF)):
                _came_from[neighbor_id] = current_id
                _g_score[neighbor_id] = tentative_g
                _f_score[neighbor_id] = _frontier_priority(neighbor_id, tentative_g)
                push_frontier(neighbor_id)
```

Cách lấy node tiếp theo từ frontier (trích từ `_pop_frontier_node()`):

```
match search_algorithm:
```

SearchAlgorithm.**BFS**:

```
while _frontier_queue_head < _search_frontier.size():  
    var candidate_id := int(_search_frontier[_frontier_queue_head])  
    _frontier_queue_head += 1  
    if _frontier_membership.has(candidate_id):  
        picked_id = candidate_id  
        break
```

SearchAlgorithm.**DFS**:

```
while not _search_frontier.is_empty():  
    var candidate_id := int(_search_frontier.pop_back())  
    if not _frontier_membership.has(candidate_id):  
        continue  
    picked_id = candidate_id  
    break
```

SearchAlgorithm.**UCS**:

```
while not _priority_frontier_heap.is_empty():  
    var entry: Array = _heap_pop()  
    var candidate_id := int(entry[1])  
    if not _frontier_membership.has(candidate_id):  
        continue  
    picked_id = candidate_id  
    break
```

Ba thuật toán phân biệt nhau hoàn toàn ở bước lấy node:

- **BFS** dùng `_frontier_queue_head` tiến dần theo chiều đầu mảng — hành vi FIFO, đảm bảo duyệt theo từng lớp.
- **DFS** dùng `pop_back()` — hành vi LIFO, luôn ưu tiên đi sâu theo nhánh hiện tại.
- **UCS** dùng `_heap_pop()` lấy node có $g(n)$ nhỏ nhất — đảm bảo mở rộng theo chi phí tích lũy tăng dần.

Cách tính chi phí: BFS và DFS mở rộng node theo thứ tự frontier, không dựa vào chi phí để chọn node. Tuy nhiên `_g_score` vẫn được cập nhật trong quá trình duyệt bằng `_distance_between()` để ghi nhận tổng chi phí đường đi sau khi tìm thấy đích. UCS sử dụng `_g_score` trực tiếp làm tiêu chí ưu tiên, đảm bảo luôn mở rộng node có chi phí tích lũy nhỏ nhất.

3.4.2. Triển khai nhóm tìm kiếm có định hướng (Greedy, A*)

Hàm heuristic: cả Greedy và A* dùng chung hàm `_heuristic()` — khoảng cách Euclid 3D từ node hiện tại đến đích, nhân với `EDGE_LENGTH_WEIGHT = 1.0`:

```
func _heuristic(a: int, b: int) -> float:
    var from_pos: Vector3 = _graph_positions[a]
    var to_pos: Vector3 = _graph_positions[b]
    return from_pos.distance_to(to_pos) * EDGE_LENGTH_WEIGHT
```

Hàm này chấp nhận được (admissible) vì khoảng cách đường thẳng trong không gian 3D không bao giờ vượt quá chi phí đường đi thực tế qua các hành lang và cầu thang.

Phân biệt Greedy và A* qua hàm ưu tiên: hàm `_frontier_priority()` thống nhất logic ưu tiên cho hai thuật toán:

```
func _frontier_priority(node_id: int, g_cost: float) -> float:
    match search_algorithm:
        SearchAlgorithm.GREEDY:
            return _heuristic(node_id, _search_goal_id)
        _: # A*
            return g_cost + _heuristic(node_id, _search_goal_id)
```

- Greedy: ưu tiên theo $h(n)$ — chỉ xét ước lượng đến đích, bỏ qua chi phí đã đi.

- **A***: ưu tiên theo $f(n) = g(n) + h(n)$ — kết hợp chi phí thực tế tích lũy và ước lượng còn lại.

Cơ chế push vào priority queue (trích từ `_push_frontier()`):

```
func _push_frontier(node_id: int) -> void:
    var is_new := not _frontier_membership.has(node_id)
    _frontier_membership[node_id] = true
    if _uses_priority_frontier():
        _heap_push(float(_f_score.get(node_id, INF)), node_id)
    else:
        _search_frontier.append(node_id)
    if is_new:
        discovered_vertex_count += 1
```

Hàm `_heap_push()` và `_heap_pop()` triển khai binary min-heap thủ công trên mảng `_priority_frontier_heap`, đảm bảo node có $f(n)$ nhỏ nhất luôn được lấy ra trước.

Điều kiện cập nhật node: trong `_step_search()`, Greedy và A* chỉ cập nhật `_came_from` và đẩy vào frontier khi `tentative_g < g_score.get(neighbor_id, INF)` — tránh xử lý lại các node đã có đường đi tốt hơn.

3.4.3. Triển khai Random-Restart Hill Climbing

Đây là kỹ thuật Random-Restart Hill Climbing thường được sử dụng để giảm xác suất mắc kẹt tại cực tiểu cục bộ bằng cách tái khởi tạo quá trình tìm kiếm theo các lựa chọn khác nhau.

Cơ chế hoạt động: Hill Climbing không duy trì frontier toàn cục. Tại mỗi bước, từ node hiện tại, thuật toán xét tất cả đỉnh kề chưa thăm, tính $h(n)$ cho từng đỉnh và chọn đỉnh có $h(n)$ nhỏ nhất (gần đích nhất theo heuristic). Trích đoạn chính từ `_run_hill_climbing_search()`:

```

var neighbors: Array = _graph_neighbors.get(current_id, [])
var candidates: Array = []
for neighbor_id_variant in neighbors:
    var neighbor_id := int(neighbor_id_variant)
    if not visited.has(neighbor_id):
        candidates.append([_heuristic(neighbor_id, _search_goal_id), neighbor_id])

if candidates.is_empty():
    break # bị kẹt — trigger restart

candidates.sort()

var pick_idx := 0
if restart_num > 0 and candidates.size() > 1:
    pick_idx = randi() % mini(3, candidates.size())

var best_id := int(candidates[pick_idx][1])
visited[best_id] = true
local_came_from[best_id] = current_id
local_g[best_id] = float(local_g.get(current_id, 0.0)) + _distance_between(current_id,
best_id)
current_id = best_id

```

Xử lý cực tiểu cục bộ: khi `candidates.is_empty()` — tức không còn đỉnh kề chưa thăm nào có $h(n)$ nhỏ hơn — thuật toán bị kẹt và kích hoạt restart. Hệ thống cho phép tối đa `MAX_RESTARTS = 6` lần:

```

const MAX_RESTARTS := 6

```

```

if not found and restart_num < MAX_RESTARTS:
    print("PathBot: Hill Climbing restart #%%d" % (restart_num + 1))

if found:
    _came_from = best_came_from
    _g_score = best_g_score
    _finalize_path()
    return

_search_finished = true
_path_found = false
print("PathBot: Hill Climbing failed after %%d restarts" % MAX_RESTARTS)

```

Ở lần restart đầu tiên, thuật toán luôn chọn đỉnh tốt nhất `pick_idx = 0`. Từ lần restart thứ hai trở đi, hệ thống chọn ngẫu nhiên trong top-3 ứng viên `pick_idx = randi() % mini(3, candidates)` để tăng khả năng thoát khỏi cực tiểu cục bộ. Nếu sau toàn bộ 6 lần restart vẫn không tìm được đường, thuật toán báo thất bại và dừng. Đây là giới hạn tiêu biểu của Hill Climbing — không có cơ chế đảm bảo tìm được đường đi tối ưu như A* hay UCS.

Sau khi thuật toán hoàn thành và `_path_node_ids` được xác định, hàm `_follow_path()` điều khiển nhân vật di chuyển tuần tự qua từng waypoint trong lộ trình với tốc độ `BOT_SPEED = 4.5 đơn vị/giây`, sử dụng `move_toward()` kết hợp `look_at()` để nhân vật luôn hướng mặt theo chiều di chuyển. Waypoint được coi là đã đến khi khoảng cách còn lại nhỏ hơn `WAYPOINT_REACHED_DIST = 0.22` đơn vị. Phần hiển thị trực quan — bao gồm các marker frontier/closed/path và debug overlay — được cập nhật định kỳ theo `visual_update_interval` để không ảnh hưởng đến hiệu năng tìm kiếm.

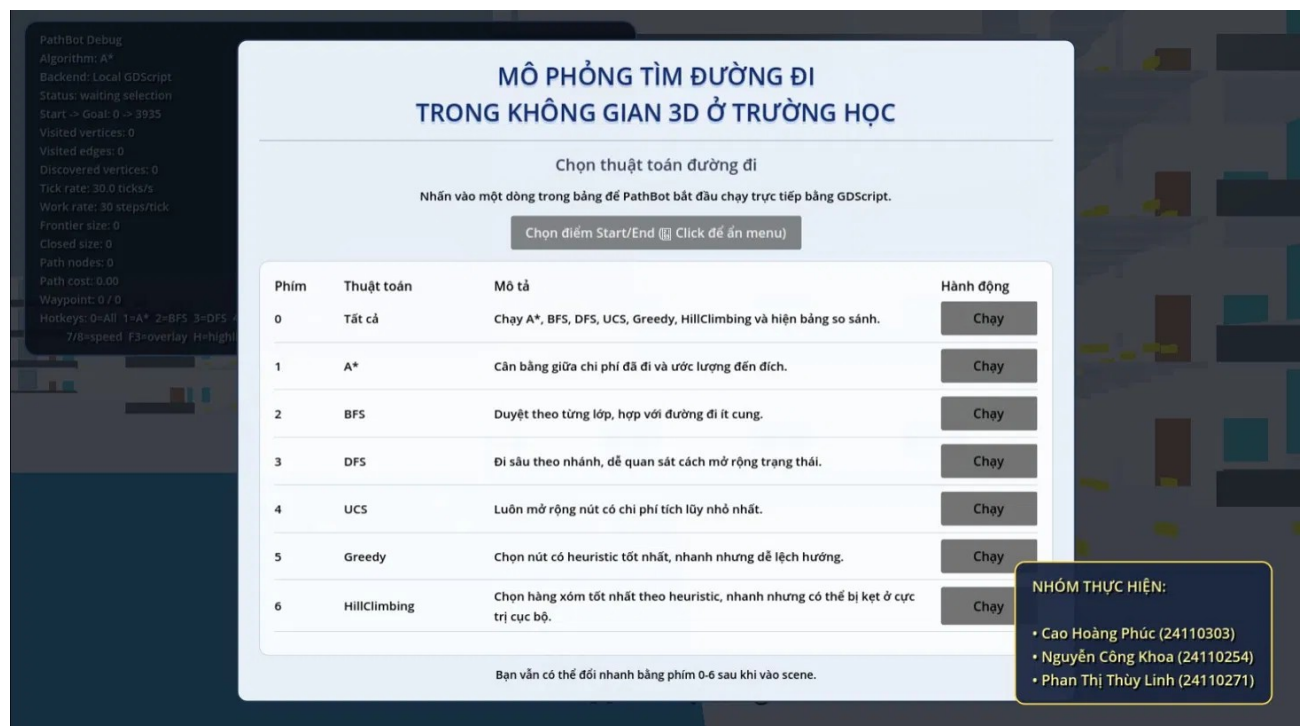
CHƯƠNG 4: THỰC NGHIỆM, ĐÁNH GIÁ, PHÂN TÍCH KẾT QUẢ

4.1. Xây dựng giao diện ứng dụng

Giao diện ứng dụng mô phỏng được xây dựng trực tiếp trong Godot Engine v4.5.2, bao gồm ba thành phần chính: màn hình menu chọn thuật toán, màn hình mô phỏng 3D và hệ thống debug overlay theo dõi quá trình tìm kiếm theo thời gian thực.

4.1.1. Màn hình menu chọn thuật toán

Khi khởi động ứng dụng, người dùng được chào đón bằng màn hình menu hiển thị bảng 6 thuật toán tìm đường có sẵn. Giao diện được thiết kế theo phong cách tối giản với nền trắng, bố cục bảng rõ ràng gồm 4 cột: Phím tắt, Tên thuật toán, Mô tả ngắn gọn và nút Chạy. Người dùng có thể kích hoạt thuật toán bằng cách nhấn nút "Chạy" tương ứng hoặc sử dụng phím tắt từ 0 đến 6 trên bàn phím. Phím 0 kích hoạt chế độ đặc biệt — chạy tuần tự tất cả 6 thuật toán và hiển thị bảng so sánh tổng hợp. Góc dưới bên phải hiển thị thông tin nhóm thực hiện. Debug overlay góc trên bên trái hiển thị trạng thái hiện tại của hệ thống ngay cả khi menu đang mở.



Hình 4.1: Menu chọn thuật toán

4.1.2. Màn hình mô phỏng 3D

Sau khi chọn thuật toán, menu tự động ẩn và người dùng quan sát trực tiếp không gian 3D khuôn viên trường học. Mô hình bao gồm các tòa nhà nhiều tầng với hành lang, cầu thang và các waypoint được đánh dấu bằng các hình khối nhỏ màu vàng phân bố khắp khuôn viên. Người dùng có thể xoay camera tự do để quan sát từ nhiều góc độ khác nhau. Hai phím hỗ trợ thêm: phím X ẩn/hiện tường tòa nhà giúp quan sát lộ trình bên trong, phím ESC quay về menu chính.



Hình 4.2: Màn hình 3D sau khi ẩn menu

4.1.3. Hệ thống trực quan hóa và Debug Overlay

Để hỗ trợ quan sát và đánh giá quá trình tìm kiếm đường đi, hệ thống tích hợp cơ chế trực quan hóa trạng thái đồ thị theo thời gian thực kết hợp với bảng thông tin Debug Overlay. Trong quá trình thực thi thuật toán, các waypoint và cạnh trong đồ thị được biểu diễn bằng các màu sắc khác nhau nhằm phản ánh trạng thái hiện tại của từng đỉnh và cung. Cụ thể:

- ☐ Marker màu vàng biểu diễn toàn bộ waypoint trong đồ thị.

- ☐ Marker màu sáng hơn biểu diễn các đỉnh thuộc tập Frontier (đang chờ được mở rộng).
- ☐ Marker màu tối hơn biểu diễn các đỉnh thuộc tập Closed Set (đã được duyệt hoàn tất).
- ☐ Các đường màu đỏ biểu diễn những cạnh đang được thuật toán khám phá.
- ☐ Nhân vật PathBot màu cam di chuyển trên lộ trình tìm được sau khi thuật toán hoàn thành.

Bên cạnh đó, hệ thống cung cấp bảng Debug Overlay tại góc trên bên trái màn hình nhằm theo dõi trạng thái hoạt động của thuật toán theo thời gian thực. Bảng thông tin bao gồm:

Tên thuật toán đang thực thi.

- ☐ Trạng thái hiện tại của hệ thống (Searching, Path Found, Running All).
- ☐ Cặp đỉnh bắt đầu và đích (Start → Goal).
- ☐ Số lượng đỉnh đã thăm (Visited Vertices).
- ☐ Số lượng cạnh đã duyệt (Visited Edges).
- ☐ Số lượng đỉnh đã được khám phá (Discovered Vertices).
- ☐ Tốc độ xử lý của hệ thống (Tick Rate).
- ☐ Kích thước tập Frontier và Closed Set.
- ☐ Số lượng nút trên đường đi tìm được.
- ☐ Tổng chi phí đường đi (Path Cost).

Thông qua cơ chế trực quan hóa này, người dùng có thể quan sát rõ quá trình mở rộng không gian tìm kiếm của từng thuật toán, từ đó thuận tiện trong việc phân tích, so sánh

hiệu năng cũng như đặc điểm hoạt động giữa các phương pháp tìm đường khác nhau.



Hình 4.3: Minh họa giao diện trực quan hóa khi thuật toán DFS đang được thực thi trong chế độ Running All.

4.2. Hướng dẫn thực thi phần mềm mô phỏng

4.2.1. Yêu cầu hệ thống

Ứng dụng được triển khai dưới dạng web, không yêu cầu cài đặt phần mềm. Người dùng chỉ cần trình duyệt web hiện đại (Chrome, Firefox, Edge) và truy cập trực tiếp tại địa chỉ:

<https://godot.caohoangphuc.id.vn/>

4.2.2. Các bước thực hiện

Bước 1 — Truy cập ứng dụng

Mở trình duyệt và truy cập đường link trên. Ứng dụng tải và hiển thị màn hình menu chọn thuật toán với bảng 6 thuật toán sẵn có.

Bước 2 — Chọn điểm Start và End

Nhấn vào nút "Chọn điểm Start/End" để ẩn menu và vào chế độ chọn điểm. Trong không gian 3D, nhấp chuột trái vào vị trí bất kỳ trên khuôn viên trường để đặt điểm xuất

phát và chuột phải vào điểm bất kì để chọn điểm đích. Hệ thống tự động snap điểm được chọn về waypoint gần nhất trong đồ thị. Nhấn "ESC" để quay lại menu sau khi đã chọn xong.



Hình 4.4: Chọn thuật toán đường đi

Bước 3 — Chọn thuật toán

Từ menu chính, người dùng có hai cách chọn thuật toán:

- ☐ Nhấn nút "Chạy" tương ứng trong bảng
- ☐ Sử dụng phím tắt từ 1 đến 6 trên bàn phím

Để chạy và so sánh tất cả 6 thuật toán cùng lúc, nhấn phím 0.

Bước 4 — Quan sát quá trình tìm kiếm

Sau khi chọn thuật toán, menu tự ẩn và quá trình tìm kiếm bắt đầu ngay lập tức. Debug overlay góc trên bên trái (Hình 4.5) cập nhật theo thời gian thực với các thông tin:

- ☐ **Algorithm:** tên thuật toán đang chạy
- ☐ **Status:** trạng thái hiện tại (searching / path found / failed)
- ☐ **Start → Goal:** cặp node xuất phát và đích
- ☐ **Visited vertices / edges:** số đỉnh và cạnh đã duyệt
- ☐ **Frontier size / Closed size:** kích thước frontier và tập đã đóng
- ☐ **Path nodes / Path cost:** số node và chi phí đường đi tìm được

```

PathBot Debug
Algorithm: HillClimbing
Backend: Local GDScript
Status: waiting selection
Start -> Goal: 666 -> 1117
Visited vertices: 99
Visited edges: 205
Discovered vertices: 205
Tick rate: 30.0 ticks/s
Work rate: 30 steps/tick
Frontier size: 0
Closed size: 0
Path nodes: 0
Path cost: 0.00
Waypoint: 0 / 0
Hotkeys: 0=All 1=A* 2=BFS 3=DFS 4=UCS 5=Greedy 6=Hillclimbing
7/8=speed F3=overlay H=highlight

```

Hình 4.5: Giao diện hiển thị thông số Debug (Debug Overlay) khi thuật toán hoạt động

Bước 5 — Xem kết quả

Khi thuật toán hoàn thành, nhân vật PathBot tự động di chuyển theo lộ trình tìm được. Nếu chạy chế độ so sánh (phím 0), bảng so sánh tổng hợp hiển thị tự động sau khi tất cả 6 thuật toán hoàn thành.

4.2.3. Bảng phím tắt

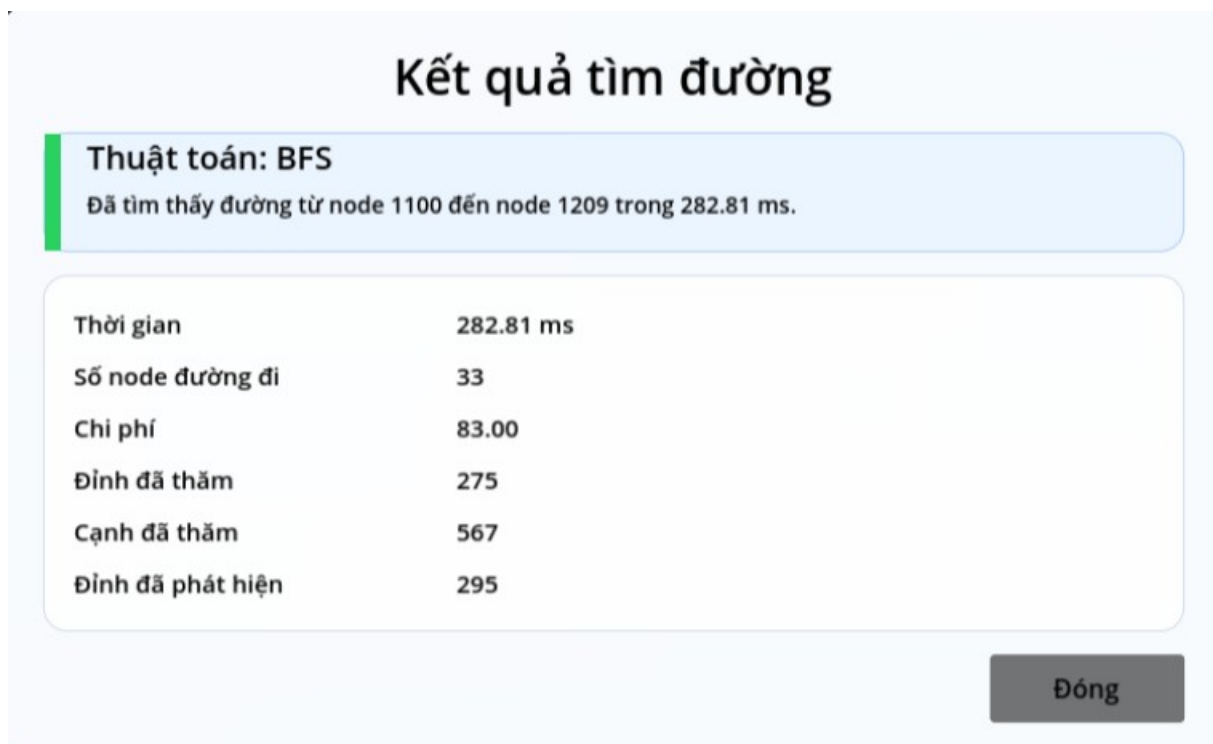
Phím	Chức năng
0	Chạy tất cả 6 thuật toán và hiển thị bảng so sánh
1	Chạy A*
2	Chạy BFS
3	Chạy DFS
4	Chạy UCS
5	Chạy Greedy
6	Chạy HillClimbing

Phím	Chức năng
7 / 8	Giảm / tăng tốc độ tìm kiếm
F3	Bật / tắt debug overlay
H	Bật / tắt highlight toàn bộ waypoint
X	Ẩn / hiện tường tòa nhà
ESC	Quay về menu chính

4.3. Kết quả sau khi chạy thuật toán

Các thực nghiệm trong mục này được thực hiện trên kịch bản di chuyển từ node 1100 đến node 1209, nhằm đánh giá hành vi và hiệu năng của từng thuật toán trong nhóm tìm kiếm mù trên cùng một cấu hình đồ thị.

4.3.1 Kết quả thuật toán tìm kiếm mù (BFS, DFS, UCS)



Hình 4.6: Kết quả thực thi thuật toán BFS

Thuật toán BFS đã tìm thành công đường đi từ node 1100 đến node 1209 trong thời gian 282,81 ms. Kết quả thu được gồm 33 node với tổng chi phí đường đi là 83. Trong quá trình tìm kiếm, thuật toán đã duyệt 275 đỉnh, 567 cạnh và khám phá tổng cộng 295 đỉnh trước khi xác định được đích.

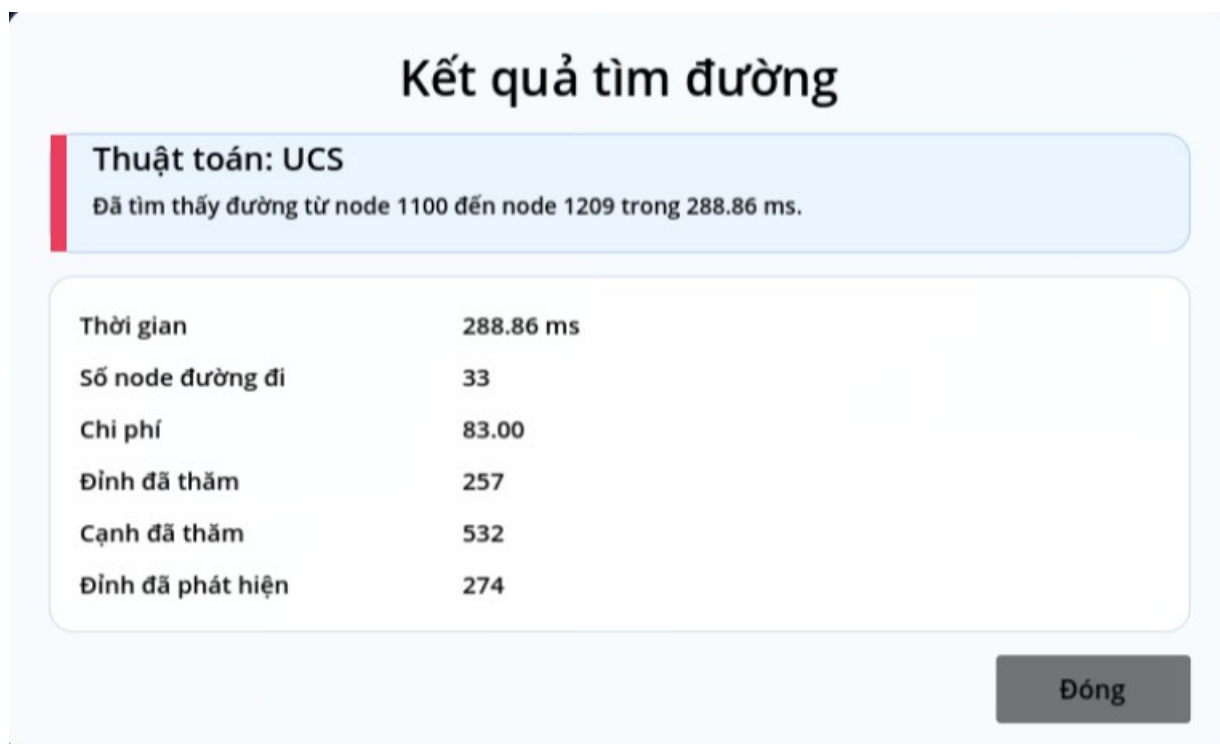
BFS hoạt động theo nguyên tắc mở rộng các đỉnh theo từng mức độ sâu, nhờ đó đảm bảo tìm được đường đi ngắn nhất trong đồ thị không trọng số. Kết quả thực nghiệm cho thấy BFS tìm được lời giải với số lượng đỉnh mở rộng tương đối thấp và thời gian xử lý nhanh. Tuy nhiên, do phải duyệt đồng thời nhiều nhánh ở cùng một mức nên thuật toán có thể tiêu tốn nhiều bộ nhớ hơn khi áp dụng trên các đồ thị có kích thước lớn.



Hình 4.7: Kết quả thực thi thuật toán DFS

Thuật toán DFS đã tìm thành công đường đi từ node 1100 đến node 1209 trong thời gian 8863,68 ms. Kết quả thu được gồm 33 node với tổng chi phí đường đi là 83. Trong quá trình tìm kiếm, thuật toán đã duyệt 6162 đỉnh, 13292 cạnh và khám phá tổng cộng 6167 đỉnh trước khi xác định được đích.

Do đặc điểm ưu tiên mở rộng theo chiều sâu, DFS thường đi sâu vào một nhánh của đồ thị trước khi quay lui để khám phá các nhánh khác. Điều này làm cho số lượng đỉnh và cạnh được duyệt tăng đáng kể, dẫn đến thời gian xử lý cao hơn so với các thuật toán có sử dụng thông tin định hướng. Mặc dù tìm được đường đi đến đích, DFS không bảo đảm tìm được lời giải tối ưu và hiệu quả tìm kiếm phụ thuộc nhiều vào thứ tự mở rộng các đỉnh trong đồ thị.

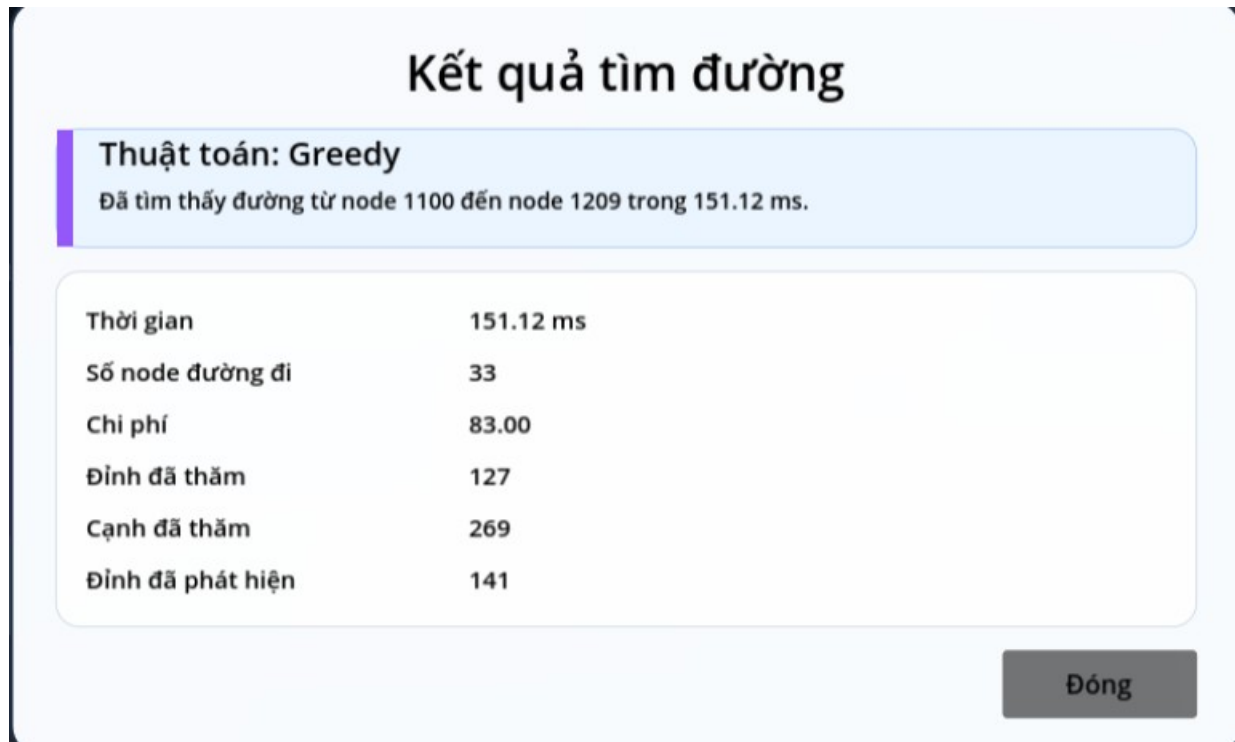


Hình 4.8: Kết quả thực thi thuật toán UCS

Thuật toán UCS đã tìm thành công đường đi từ node 1100 đến node 1209 trong thời gian 288,86 ms. Kết quả thu được gồm 33 node với tổng chi phí đường đi là 83. Trong quá trình tìm kiếm, thuật toán đã duyệt 257 đỉnh, 532 cạnh và khám phá tổng cộng 274 đỉnh trước khi xác định được đích.

UCS hoạt động theo nguyên tắc luôn mở rộng đỉnh có tổng chi phí tích lũy nhỏ nhất từ điểm xuất phát. Nhờ đó, thuật toán bảo đảm tìm được đường đi tối ưu trong đồ thị có trọng số không âm. Kết quả thực nghiệm cho thấy UCS đạt hiệu quả tốt với số lượng đỉnh và cạnh được mở rộng thấp hơn BFS, đồng thời vẫn bảo đảm chất lượng lời giải tối ưu.

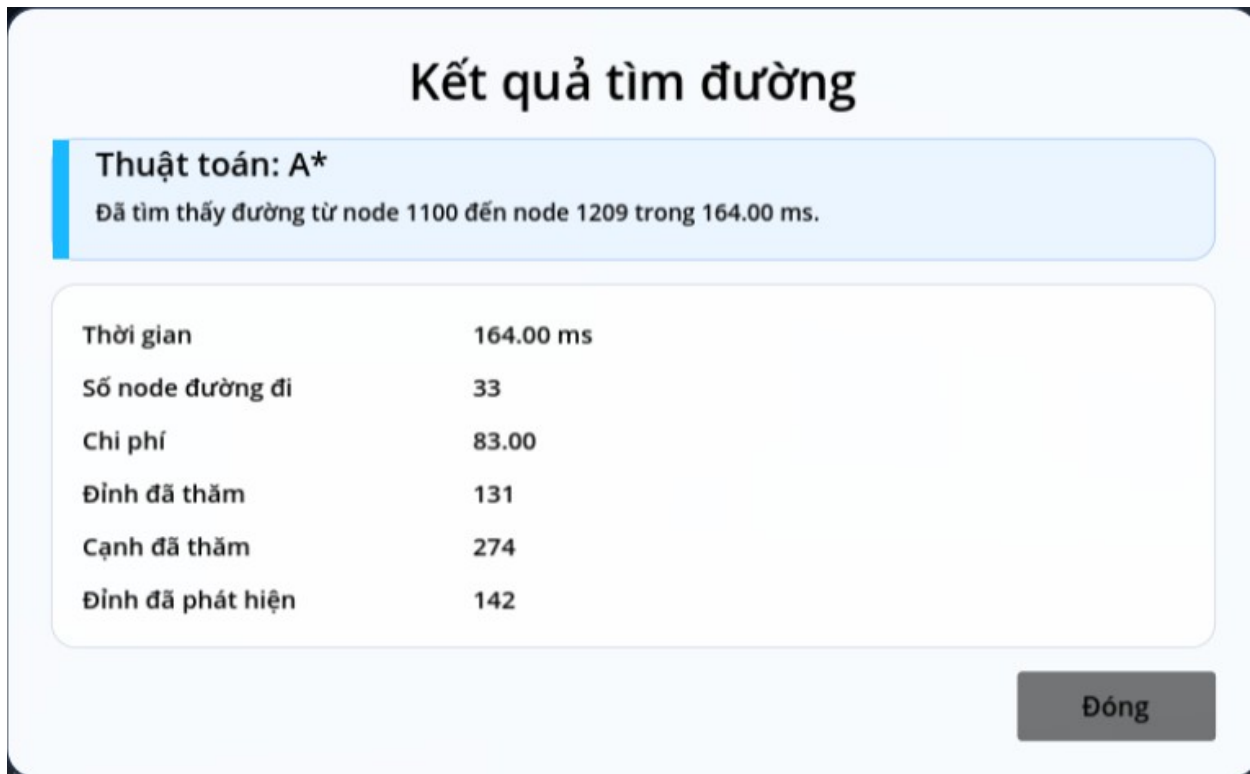
4.3.2. Kết quả thuật toán tìm kiếm có định hướng (Greedy, A*)



Hình 4.9: Kết quả thực thi thuật toán Greedy Best-First Search

Thuật toán Greedy Best-First Search đã tìm thành công đường đi từ node 1100 đến node 1209 trong thời gian 151,12 ms. Kết quả thu được gồm 33 node với tổng chi phí đường đi là 83. Trong quá trình tìm kiếm, thuật toán đã duyệt 127 đỉnh, 269 cạnh và khám phá tổng cộng 141 đỉnh trước khi xác định được đích.

Greedy Best-First Search hoạt động dựa trên hàm heuristic để lựa chọn đỉnh có khoảng cách ước lượng đến đích nhỏ nhất. Nhờ tập trung vào các khu vực được đánh giá là gần mục tiêu, thuật toán giảm đáng kể số lượng đỉnh cần mở rộng và đạt thời gian thực thi thấp. Kết quả thực nghiệm cho thấy Greedy là thuật toán có tốc độ tìm kiếm nhanh trong môi trường thử nghiệm, tuy nhiên không bảo đảm tìm được lời giải tối ưu trong mọi trường hợp.



*Hình 4.10: Kết quả thực thi thuật toán A**

Thuật toán A* đã tìm thành công đường đi từ node 1100 đến node 1209 trong thời gian 164 ms. Kết quả cho thấy đường đi tìm được gồm 33 node với tổng chi phí là 83. Trong quá trình tìm kiếm, thuật toán đã mở rộng 131 đỉnh, duyệt 274 cạnh và khám phá tổng cộng 142 đỉnh trước khi xác định được lời giải.

Các số liệu thu được cho thấy A* đạt hiệu quả tốt trong việc giảm số lượng đỉnh cần duyệt nhờ sử dụng hàm heuristic để định hướng quá trình tìm kiếm. Việc kết hợp giữa chi phí thực tế đã đi và chi phí ước lượng đến đích giúp thuật toán tìm được đường đi tối ưu với thời gian xử lý thấp, phù hợp với môi trường đồ thị 3D có số lượng waypoint lớn.

4.3.3 Kết quả thuật toán tìm kiếm cục bộ (Hill Climbing)



Hình 4.11: Hill Climbing không tìm được đường đi đến đích

Trong thí nghiệm với điểm xuất phát tại node 1100 và điểm đích tại node 1209, thuật toán Hill Climbing không tìm được lời giải. Trạng thái hệ thống hiển thị *failed*, đồng thời số node đường đi và chi phí đường đi đều bằng 0. Trong quá trình tìm kiếm, thuật toán đã duyệt 110 đỉnh, 232 cạnh và khám phá 232 đỉnh trước khi dừng lại.

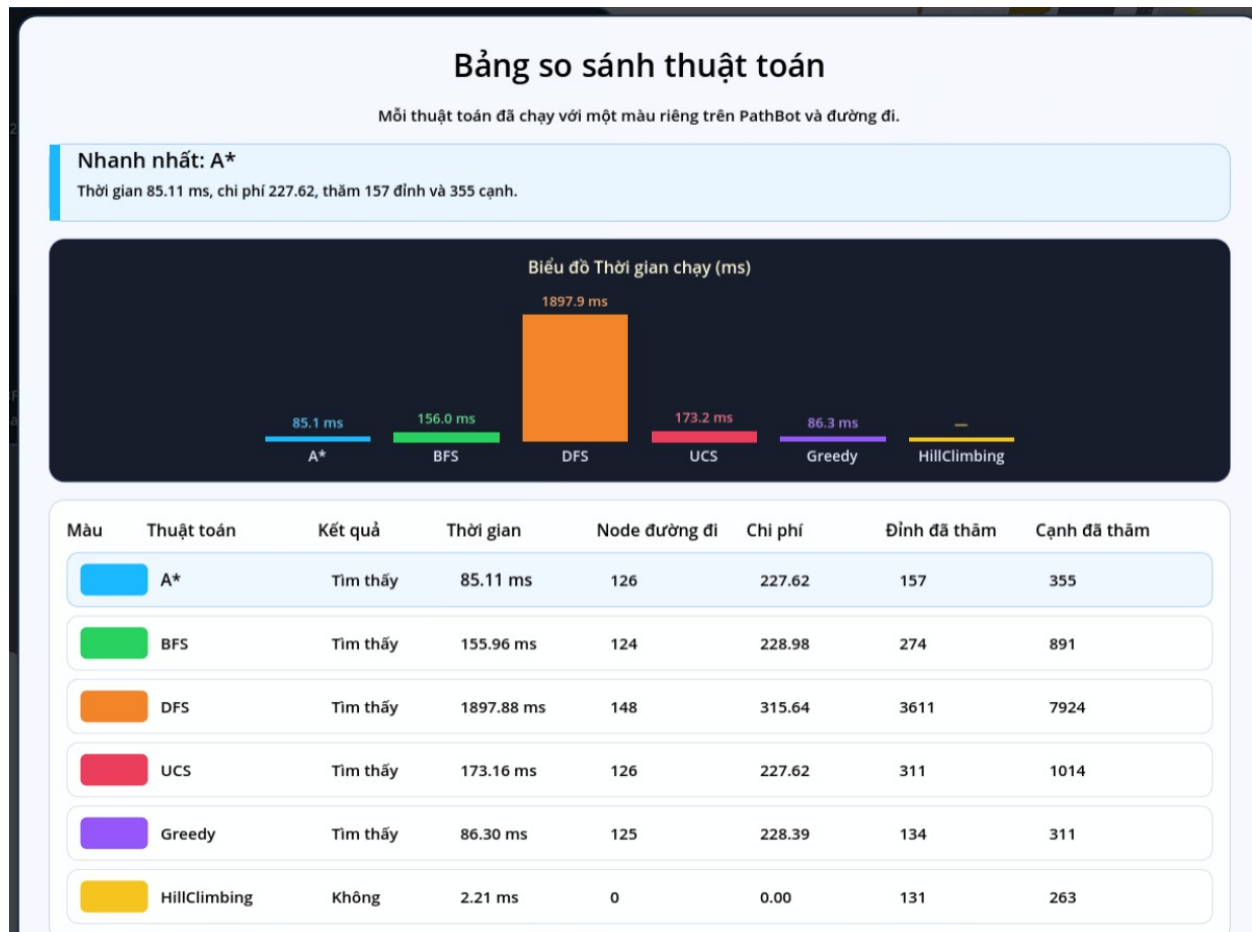
Nguyên nhân: xuất phát từ đặc điểm của Hill Climbing là chỉ lựa chọn trạng thái lân cận có giá trị heuristic tốt hơn trạng thái hiện tại. Khi gặp một vị trí mà tất cả các đỉnh lân cận đều có giá trị heuristic kém hơn, thuật toán sẽ không thể tiếp tục mở rộng mặc dù chưa đến đích. Hiện tượng này được gọi là cực trị địa phương (local optimum).

Kết quả thực nghiệm: cho thấy Hill Climbing có thể hoạt động nhanh trong một số trường hợp, tuy nhiên không bảo đảm tìm được lời giải và dễ thất bại trong môi trường 3D có nhiều tầng, hành lang và cầu thang như mô hình được xây dựng trong đề tài.

4.4. So sánh các thuật toán

4.4.1. Kịch bản 1 — Di chuyển cùng tầng

Mục tiêu: Đánh giá hiệu năng của 6 thuật toán khi di chuyển giữa hai điểm trên cùng một tầng lầu, không có sự thay đổi độ cao (trục Y).



Hình 4.12: Bảng so sánh hiệu năng 6 thuật toán: Kịch bản di chuyển cùng tầng

Nhận xét:

Trong kịch bản di chuyển cùng tầng, 5 trong 6 thuật toán tìm thấy đường đi thành công.

A* là thuật toán hiệu quả nhất tổng thể: thời gian 85,11 ms, chi phí tối ưu 227,62, chỉ duyệt 157 đỉnh và 355 cạnh. Sự kết hợp giữa chi phí thực tế $g(n)$ và heuristic $h(n)$ giúp A* vừa nhanh vừa đảm bảo tối ưu.

Greedy có thời gian gần bằng A* (86,30 ms) và duyệt ít đỉnh nhất (134 đỉnh), nhưng chi phí đường đi 228,39 cao hơn A* và UCS — do chỉ dựa vào heuristic mà không tính chi phí đã đi, dẫn đến lộ trình không hoàn toàn tối ưu.

UCS đạt chi phí tối ưu bằng A* (227,62) nhưng chậm hơn gấp đôi (173,16 ms) và duyệt nhiều hơn (311 đỉnh) — do không có heuristic dẫn hướng, phải mở rộng đồng đều theo mọi hướng.

BFS tìm được đường đi 124 node với chi phí 228,98 — gần tối ưu nhưng không bằng A* và UCS vì không phân biệt trọng số cạnh. Duyệt 274 đỉnh và 891 cạnh, tốn bộ nhớ hơn các thuật toán có heuristic.

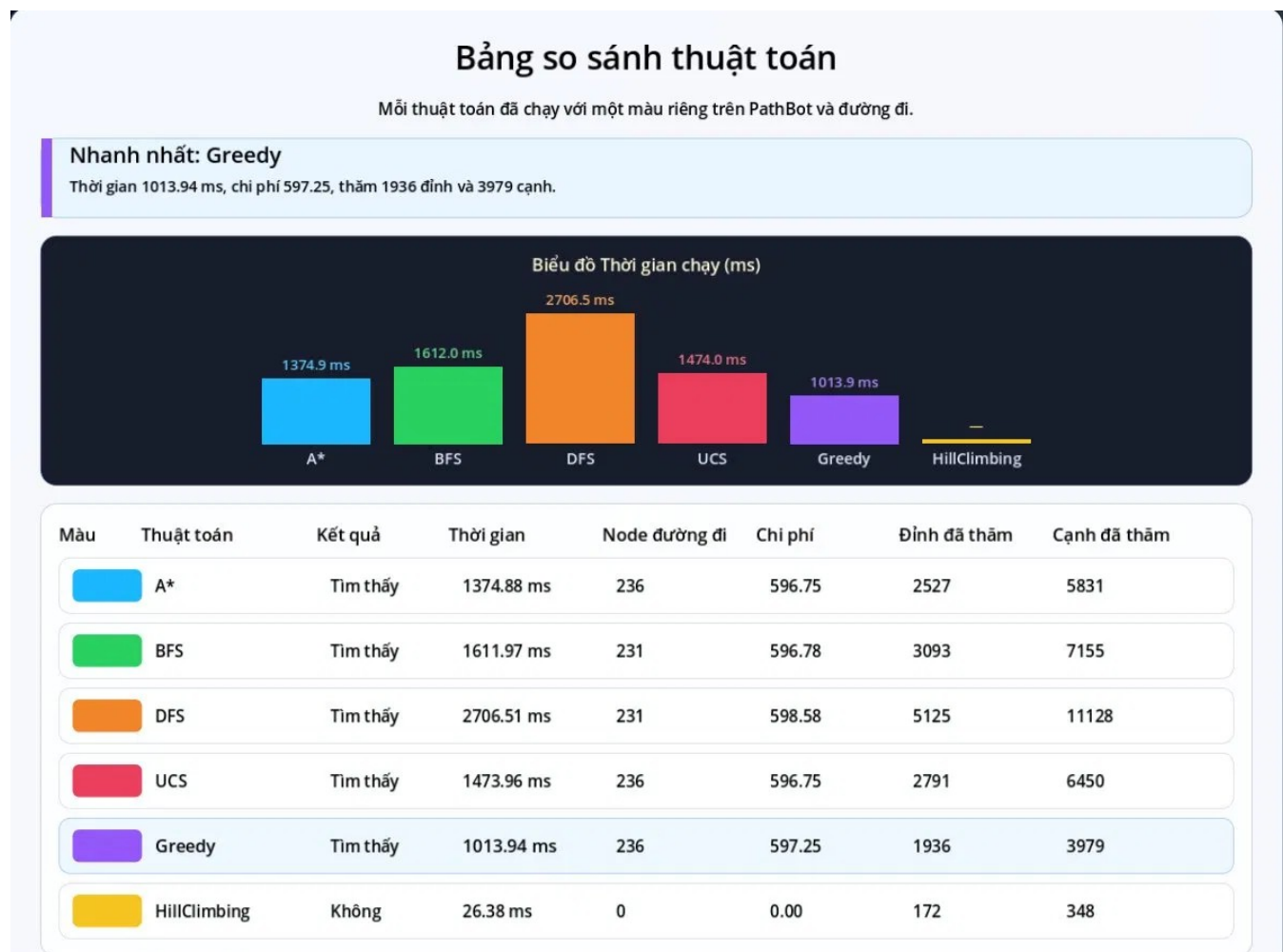
DFS là thuật toán kém hiệu quả nhất trong kịch bản này: thời gian lên đến 1897,88 ms, duyệt 3611 đỉnh và 7924 cạnh — gấp hơn 20 lần so với A*. Đường đi dài nhất (148 node, chi phí 315,64) do đặc tính LIFO khiến thuật toán đi vòng vèo qua nhiều nhánh không liên quan trước khi tìm thấy đích.

HillClimbing thất bại ngay trong kịch bản cùng tầng (0 node đường đi) mặc dù đã duyệt 131 đỉnh trong 2,21 ms. Thuật toán bị kẹt ở cực tiểu cục bộ và cơ chế restart 6 lần không đủ để thoát ra trong cấu trúc đồ thị waypoint của dự án. Đây là giới hạn cố hữu của Hill Climbing khi áp dụng trên đồ thị lớn có nhiều ngõ cụt.

4.4.2. Kịch bản 2 — Di chuyển xuyên tầng

Mục tiêu: Đánh giá khả năng xử lý của 6 thuật toán khi di chuyển giữa hai điểm thuộc các tầng khác nhau, đòi hỏi lộ trình phải đi qua cầu thang — nút thắt cổ chai với

chi phí leo tầng được tính theo cơ chế phạt độ cao (ascent penalty)



Hình 4.13: Bảng so sánh hiệu năng 6 thuật toán: Kịch bản di chuyển xuyên tầng

Nhận xét:

So với kịch bản 1, kịch bản xuyên tầng cho thấy sự gia tăng đáng kể về chi phí và thời gian tính toán ở tất cả các thuật toán — phản ánh đúng độ phức tạp cao hơn của bài toán đa tầng. chi phí tối ưu tăng từ 227,62 lên 596,75 do lộ trình bắt buộc phải đi qua cầu thang với hệ số phạt leo tầng bằng 4 lần độ cao chênh lệch theo công thức

$$w(u, v) = d(u, v) + 4 * ascent(u, v)$$

đã trình bày ở Chương 3.

Greedy là thuật toán nhanh nhất (1013,94 ms) với số đỉnh duyệt thấp nhất (1936 đỉnh, 3979 cạnh). Heuristic Euclid 3D dẫn hướng hiệu quả về phía đích ngay cả khi phải

đổi tầng, giúp Greedy thu hẹp không gian tìm kiếm đáng kể. Tuy nhiên chi phí đường đi 597,25 cao hơn A* và UCS (596,75) — cho thấy lộ trình chưa hoàn toàn tối ưu.

A* tiếp tục đảm bảo chi phí tối ưu (596,75) với thời gian 1374,88 ms và 2527 đỉnh đã duyệt — hiệu quả thứ hai sau Greedy. Sự kết hợp $g(n) + h(n)$ giúp A* không bị dẫn lạc bởi các nhánh đồ thị có heuristic tốt nhưng chi phí thực tế cao.

UCS đạt chi phí tối ưu bằng A* (596,75) nhưng duyệt nhiều hơn (2791 đỉnh, 6450 cạnh) và chậm hơn (1473,96 ms). Không có heuristic dẫn hướng, UCS phải mở rộng đồng đều theo mọi hướng — bất lợi rõ rệt trong không gian đồ thị lớn xuyên tầng.

BFS tìm được đường đi 231 node với chi phí 596,78 — gần tối ưu nhưng không bằng A* và UCS do không phân biệt trọng số cạnh. Duyệt 3093 đỉnh và 7155 cạnh, tốn bộ nhớ hơn đáng kể so với các thuật toán có heuristic.

DFS tiếp tục là thuật toán kém hiệu quả nhất: thời gian 2706,51 ms, duyệt 5125 đỉnh và 11128 cạnh. Đáng chú ý là DFS tìm được đường đi 231 node với chi phí 598,58 — gần bằng BFS về số node nhưng chi phí cao hơn do lộ trình không tối ưu. Trong môi trường xuyên tầng với nhiều nhánh phức tạp, đặc tính LIFO của DFS dẫn đến việc khám phá quá nhiều nhánh không cần thiết trước khi tìm đến cầu thang.

HillClimbing thất bại hoàn toàn (0 node đường đi, 26,38 ms). Đây là minh chứng rõ nhất cho giới hạn của thuật toán tìm kiếm cục bộ trong môi trường đa tầng: khi nhân vật đứng ngay bên dưới điểm đích ở tầng trên, hàm heuristic Euclid 3D cho thấy vị trí hiện tại đã rất gần đích, nhưng tất cả các đỉnh kề đều nằm trên hành lang cùng tầng — di chuyển ngang ra cầu thang làm tăng $h(n)$, khiến HillClimbing từ chối thực hiện và bị kẹt. Cơ chế restart 6 lần cũng không đủ để thoát khỏi cực tiểu cục bộ trong cấu trúc đồ thị phức tạp này.

4.4.3. Nhận xét tổng hợp

Qua hai kịch bản thực nghiệm, một số kết luận chung có thể rút ra:

A* là thuật toán cân bằng nhất — luôn đảm bảo chi phí tối ưu ở cả hai kịch bản, thời gian và số đỉnh duyệt ở mức trung bình thấp. Đây là lựa chọn phù hợp nhất cho bài toán tìm đường thực tế trong môi trường trường học.

Greedy nhanh nhất về thời gian nhưng không đảm bảo tối ưu — phù hợp khi ưu tiên tốc độ hơn chất lượng đường đi.

UCS đảm bảo tối ưu như A^* nhưng chậm hơn đáng kể do thiếu heuristic — ít phù hợp với đồ thị lớn.

BFS ổn định, gần tối ưu nhưng tốn bộ nhớ — phù hợp khi các cạnh có trọng số tương đương nhau.

DFS kém hiệu quả nhất trong cả hai kịch bản — không phù hợp cho bài toán tìm đường thực tế có trọng số.

HillClimbing thất bại ở cả hai kịch bản — xác nhận giới hạn của tìm kiếm cục bộ trên đồ thị waypoint lớn, không phù hợp cho bài toán này.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Sau thời gian nghiên cứu và hiện thực hóa, đề tài "*Mô phỏng tìm đường đi trong không gian 3D ở trường học*" đã hoàn thành các mục tiêu tổng quát và cụ thể đã đề ra.

Về mục tiêu tổng quát, nhóm đã xây dựng thành công ứng dụng mô phỏng không gian 3D trường học tích hợp công cụ kiểm thử và so sánh trực quan hiệu năng của các thuật toán tìm đường AI, triển khai hoàn chỉnh trên nền tảng Godot Engine v4.5.2 và có thể truy cập trực tiếp qua trình duyệt web.

Về các mục tiêu cụ thể:

- Đã xây dựng thành công mô hình 3D khuôn viên trường học đa tầng bao gồm các tòa nhà, hành lang, cầu thang và phòng học, phản ánh đúng cấu trúc không gian thực tế. Môi trường được số hóa thành mạng lưới waypoint quy mô lớn với 6.298 đỉnh và 6.746 cạnh, đảm bảo biểu diễn đầy đủ các lựa chọn di chuyển trong khuôn viên. Bài toán chênh lệch độ cao (trục Y) được xử lý thông qua hệ số cản k, giúp trọng số đồ thị phản ánh đúng nỗ lực di chuyển thực tế giữa các tầng lầu.
- Đã hiện thực hóa thành công 6 thuật toán tìm kiếm bằng ngôn ngữ GDScript trên Godot Engine v4.5.2, bao gồm: BFS và DFS (nhóm tìm kiếm mù), UCS (tối ưu chi phí), Greedy và A* (tìm kiếm có định hướng), Hill Climbing (tìm kiếm cục bộ). Mỗi thuật toán được cài đặt độc lập theo module, tích hợp cơ chế xử lý đặc thù cho môi trường 3D (binary min-heap cho UCS/A*, cơ chế restart cho Hill Climbing).
- Đã xây dựng hệ thống so sánh thuật toán dựa trên các chỉ số thực nghiệm đo lường trực tiếp: thời gian tính toán, số lượng đỉnh và cạnh đã duyệt, độ dài đường đi và tổng chi phí lộ trình. Kết quả thực nghiệm trên hai kịch bản (di chuyển cùng tầng và xuyên tầng) cung cấp minh chứng rõ ràng: A* là giải pháp toàn diện và tối ưu nhất cho bài toán tìm đường 3D có trọng số, đồng thời làm rõ giới hạn cố hữu của thuật toán tìm kiếm cục bộ (Hill Climbing) khi áp dụng vào không gian kiến trúc đa tầng phức tạp.

5.2. Hạn chế của đề tài

Mặc dù hệ thống đã đáp ứng được các mục tiêu đề ra và cho thấy khả năng mô phỏng hiệu quả các thuật toán tìm đường trong không gian ba chiều, vẫn còn tồn tại một số hạn chế cần được xem xét.

1. Hạn chế về dữ liệu và mô hình môi trường

Mô hình đồ thị được xây dựng theo dạng tĩnh (Static Graph), trong đó cấu trúc đỉnh và cạnh không thay đổi trong suốt quá trình mô phỏng. Vì vậy, hệ thống chưa phản ánh được các tình huống thực tế có tính động như sự xuất hiện của đám đông sinh viên, hành lang tạm thời bị phong tỏa hoặc phòng học bị khóa.

Bên cạnh đó, việc di chuyển giữa các tầng hiện mới được mô hình hóa thông qua cầu thang bộ với cơ chế điều chỉnh trọng số. Các phương tiện di chuyển khác như thang máy chưa được tích hợp, do đó hệ thống chưa xét đến các yếu tố như thời gian chờ hoặc năng lực phục vụ của thang máy trong bài toán tìm đường.

2. Hạn chế về thuật toán

Kết quả thực nghiệm cho thấy thuật toán Hill Climbing có thể đạt được thời gian xử lý rất nhanh trong nhiều trường hợp. Tuy nhiên, do đặc trưng của phương pháp tìm kiếm cục bộ, thuật toán vẫn có nguy cơ rơi vào các cực tiểu cục bộ (local minima).

Mặc dù hệ thống đã áp dụng cơ chế khởi động lại ngẫu nhiên với tối đa sáu lần thử, một số trường hợp đặc biệt trong không gian ba chiều vẫn khiến thuật toán không tìm được đường đi đến đích. Hiện tượng này thường xảy ra khi đỉnh đích nằm ở tầng trên nhưng vị trí hiện tại lại nằm ngay bên dưới đích, khiến thuật toán bị mắc kẹt tại các khu vực lân cận có giá trị heuristic tốt nhưng không dẫn đến lời giải tối ưu.

3. Hạn chế về đánh giá hiệu năng

Quá trình đánh giá hiện tại chủ yếu tập trung vào các chỉ số như thời gian thực thi, số lượng đỉnh đã duyệt và chi phí đường đi. Tuy nhiên, hệ thống chưa tích hợp cơ chế theo dõi và phân tích mức sử dụng bộ nhớ trong quá trình thực thi thuật toán.

Do đó, việc đánh giá hiệu năng mới chỉ phản ánh khía cạnh tốc độ xử lý, trong khi các yếu tố quan trọng khác như mức tiêu thụ RAM hoặc khả năng mở rộng trên các đồ thị có quy mô lớn vẫn chưa được khảo sát đầy đủ.

5.3. Định hướng phát triển

Dựa trên những hạn chế đã phân tích, nhóm đề xuất các hướng phát triển tiếp theo nhằm hoàn thiện và đưa đề tài vào ứng dụng thực tiễn.

Thứ nhất, nâng cấp thuật toán: Nghiên cứu tích hợp các giải thuật tìm đường động như D* hoặc LPA* để có khả năng tự động cập nhật lại lộ trình khi xuất hiện vật cản bất ngờ mà không cần tính toán lại toàn bộ đồ thị. Đồng thời, có thể áp dụng Simulated Annealing (Luyện kim mô phỏng) để thay thế Hill Climbing, giúp thuật toán chấp nhận các bước đi tạm thời "xấu hơn" nhằm thoát khỏi cực tiểu cục bộ.

Thứ hai, đa dạng hóa tính năng: Mở rộng thêm cơ chế chọn tiêu chí tìm đường tùy chỉnh theo nhu cầu đặc thù, chẳng hạn chế độ tìm đường ưu tiên cho người khuyết tật (chỉ sử dụng thang máy và đường dốc, bỏ qua cầu thang bộ) hoặc lộ trình có mái che phục vụ những ngày thời tiết xấu.

Thứ ba, ứng dụng thực tiễn: Chuyển đổi nền tảng mô phỏng hiện tại thành ứng dụng di động dành riêng cho sinh viên và khách tham quan khuôn viên trường. Việc kết hợp mạng lưới waypoint 3D với công nghệ Thực tế tăng cường (AR) sẽ tạo ra công cụ dẫn đường trực quan, hiển thị mũi tên chỉ dẫn theo thời gian thực ngay trên màn hình camera điện thoại của người dùng.

TÀI LIỆU THAM KHẢO

- [1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] Godot Engine Documentation. (n.d.). *Godot Engine 4.x documentation*. Retrieved June 2026, from <https://docs.godotengine.org/en/stable/>
- [3] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>